

Vision-Language-Action Models for Semantic Plastic Waste Sorting with Transparent Object Perception

Méa Vittot

Student Number: H00512699

Msc Project Dissertation

School of Engineering and Physical Sciences
Electrical, Electronic & Computer Engineering
Heriot-Watt University

Supervisor: Dr. Maria Koskinopoulou

Date: August 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aim and Objectives	1
1.3	Contributions	2
1.4	Dissertation Structure	2
2	Literature Review	3
2.1	Introduction	3
2.2	Classical Plastic Sorting: Sensing and Classification	3
2.3	Semantic Scene Understanding for Robotic Sorting	5
2.4	Vision-Language-Action Models	6
2.4.1	Spatial Grounding: A Documented Weakness	9
2.5	Fine-Tuning Practice: Recipes, Forgetting, and What Adaptation Preserves	9
2.6	Synthetic Data and Its Hazards	10
2.7	Technical Gaps for VLA-Based Plastic Sorting	11
2.8	Research Gap, Aim, and Questions	13
3	Methodology	14
3.1	Work Package 1: Synthetic Data Generation (ReSort-IT)	14
3.1.1	Source Assets	14
3.1.2	Grasp Target Derivation	16
3.1.3	Compositing and Domain Randomisation	17
3.1.4	Real-Frame Mixing and Native Resolution	18
3.1.5	Data Partitioning	19
3.1.6	Action Representation and Tokenisation	19
3.2	Work Package 2: Perception Front-End	19
3.2.1	Network Architecture	19
3.2.2	Loss and Optimisation	20
3.2.3	Depth Reconstruction Solver	21
3.3	Work Package 3: VLA Fine-Tuning	22
3.3.1	Base Model	22
3.3.2	Adapting Models Parameters with LoRA	22
3.3.3	Training Protocol	22
3.3.4	Fine-Tuning Configurations: A Three-Run Sequence	23

3.4	Evaluation Methodology	24
3.4.1	Perception Metrics	25
3.4.2	Depth Reconstruction Ablation	25
3.4.3	Policy Metrics	25
3.4.4	Instruction-Sensitivity Protocol	26
3.4.5	Base-Model Control	27
3.4.6	Real-World Transfer: ZeroWaste and Perception Hit-Precision	27
3.4.7	Runtime Profiling	27
3.5	Implementation and Hardware	27
3.6	Summary	28
4	Results and Analysis	29
4.1	Perception Module	29
4.2	Depth Reconstruction Ablation	30
4.2.1	Anchor-Weight Sensitivity	32
4.3	Policy Learning	33
4.3.1	Diagnosing How the Model Memorised Data	33
4.3.2	Corrected Training Dynamics	34
4.3.3	Spatial Accuracy	34
4.4	Grasp-Feasibility Measurement	35
4.4.1	Label Audit	35
4.4.2	Run 1 Baseline	36
4.4.3	Run 2: Effect of Data Revisions	36
4.4.4	Breakdown by Sample Type	37
4.4.5	Breakdown by Object Size	38
4.4.6	The Spatial Ceiling Holds Across Runs	38
4.5	Instruction Robustness and Semantic Grounding	39
4.6	Base-Model Control: Fine-Tuning Caused the Instruction Blindness	40
4.7	Real-World Transfer: ZeroWaste	41
4.7.1	Validation Loss Is a Misleading Selection Signal	42
4.8	End-to-End Qualitative Results	42
4.9	Runtime Profiling	42
4.10	Summary of Findings	44
5	Discussion	45
5.1	Synthetic Supervision and Spatial Grounding (RQ1)	45
5.2	Robustness Is Not Grounding	46
5.3	Perception Generalisation and Error Propagation (RQ2, RQ3)	47
5.4	From Spatial Accuracy to Grasp Feasibility (RQ4)	48
5.5	Limitations	49
5.6	Summary	49

6	Conclusions and Recommendations	51
6.1	Conclusions	51
6.2	Contributions to Knowledge and Practice	52
6.3	Recommendations and Future Work	52
6.4	Closing Remark	53

List of Figures

2.1	An example frame from the ZeroWaste dataset illustrating the extreme clutter, deformation, and contamination typical of real-world material recovery facilities. This represents the domain gap that synthetic-only VLA models must eventually cross. Figure reproduced from Bashkirova et al. [1].	5
2.2	The ClearGrasp depth reconstruction pipeline. Three parallel networks infer surface normals, transparent-object masks, and occlusion boundaries from an RGB-D input. A global optimisation step then uses these outputs to correct and complete the raw depth map. Figure reproduced from Sajjan et al. [2].	6
2.3	Spatial affordance prediction using RoboPoint. Rather than outputting bounding boxes, the vision-language model translates natural language instructions into precise 2D spatial coordinates (affordances) for robotic manipulation and navigation. Figure reproduced from Yuan et al. [3].	7
2.4	The OpenVLA model architecture. A vision encoder (fusing DINOv2 and SigLIP features) and a text instruction are mapped into a 7-billion-parameter Llama-2 backbone, which outputs 7-dimensional robot control actions. Figure reproduced from Kim et al. [4].	8
2.5	The action tokenisation scheme introduced by RT-2 and adopted by OpenVLA. The model predicts a high-level text plan followed by a sequence of discrete text tokens (numbers) representing the continuous 7-dimensional robot control actions. Figure reproduced from Zitkovich et al. [5].	8
3.1	End-to-end system architecture. The WP2 perception module provides repaired metric depth, which combines with the WP3 VLA policy’s 2D spatial tokens to yield 3D workspace coordinates via kinematics.	15
3.2	Visual comparison of a native scene versus procedurally composited over an arbitrary background. The compositing process degrades the environmental lighting interactions (refraction, caustics, and cast shadows) that serve as critical grounding cues for the VLA.	15
3.3	Intermediate representations provided by the ClearGrasp source assets. From left to right: the RGB image, the binary segmentation mask, the ground-truth surface normals, and the depth map. These isolated modalities form the foundation of the synthetic training pipeline.	16

3.4	Deriving where to grasp complex objects. (Left) The object segmentation mask. (Middle) The naive geometric centroid falls outside the physical object boundary due to the concave shape, producing an invalid grasp target. (Right) The corrected target, derived by calculating the distance transform of the mask and extracting the internal point furthest from all boundaries. .	17
3.5	Perception module architecture.	20
3.6	The depth reconstruction process under the ablation protocol (Section 3.4.2). From left to right: the RGB input; the simulated sensor failure where ground-truth depth is corrupted over transparent surfaces; the surface normals used to guide the solver; and the final reconstructed metric depth map.	21
4.1	In-distribution perception behaviour on the training category. Left: surface-normal angular-error distribution, right-skewed with its mode near 8° and a long tail beyond 40° . Centre: cumulative threshold-accuracy curve with the standard 11.25° and 22.5° thresholds marked. Right: a qualitative example (input RGB, ground-truth normals, predicted normals, predicted mask) showing accurate object localisation with smoothed normal detail. Out-of-distribution results are tabulated in Table 4.1.	30
4.2	Depth reconstruction ablation. Top rows: qualitative comparisons on three scenes (RGB input, ground-truth depth, corrupted sensor depth, inpainting baseline, Poisson with ground-truth normals, Poisson with predicted normals), with red contours marking the transparent regions; residual artefacts inside the contours are visible only in the predicted-normals condition. Bottom: RMSE box plots per method (note the outliers in the ground-truth-normals condition) and mean-RMSE bar chart.	31
4.3	Mean RMSE per reconstruction method with bootstrap 95% confidence intervals.	32
4.4	Reconstruction RMSE against anchor weight λ (log scale, error bars one standard deviation). The chosen operating point $\lambda = 100$ lies at the end of a flat plateau spanning $\lambda \in [10, 100]$	33
4.5	Policy spatial accuracy against a centre-prior baseline, matched sample-for-sample ($n = 251$). Left: Mean and median pixel distance to the ground-truth target. Right: Histogram of the model’s spatial error distribution, highlighting the subset of predictions exceeding a 100-pixel threshold.	35
4.6	Run 2 spatial L1 error (left: mean and median) and per-axis coordinate error (right) across training checkpoints. The mean is volatile due to parser contamination at some checkpoints; the median (purple) is stable and falls to ~ 21 bins by step 8,000, remaining there through 39,000 steps.	36
4.7	Grasp on-target rate by sample type (native ClearGrasp frames versus composited frames) evaluated on the Run 2 policy.	37
4.8	Run 2 grasp-target predictions by object-area tercile.	38

4.9	Run 2 grasp-target predictions. Left two: TARGET HIT (prediction on the selected object). Middle two: OTHER OBJECT (prediction on a different item in the cluster). Right two: MISS (prediction on background). Yellow contours: target object; cyan contours: all objects; red cross: prediction; green cross: ground truth.	39
4.10	Comparison of off-template parse-failure rates (red) and on-target grasp rates (blue) across the three fine-tuning configurations. Parse failures measure output formatting validity, while on-target rates measure semantic spatial grounding.	41
4.11	End-to-end pipeline outputs on four native ClearGrasp scenes (a domain shift from the policy’s composite training data). Columns: RGB input, predicted normals, reconstructed depth, VLA grasp target (red cross) with object boundaries (cyan).	43
4.12	Latency breakdown across different stages of the pipeline.	43

List of Tables

2.1	Summary of key approaches in automated plastic sorting and robotic manipulation.	12
3.1	Final (Run 2) dataset composition by split and sample type.	18
3.2	Fine-tuning configurations. Rows above the rule are held constant across runs; rows below are varied deliberately to test the cause of the policy’s failure.	24
4.1	Perception accuracy in and out of distribution. Angular errors in degrees over object pixels; threshold columns give the fraction of pixels with error below the stated angle. ID = in-distribution (overlaps training data); OOD = held-out category.	29
4.2	Depth reconstruction error over the corrupted region (100 images, half resolution, corrected camera focal length $f_x = f_y = 693$ at 960×540). RMSE with bootstrap 95% confidence intervals from 10^4 resamples; MAE and mean relative error given for completeness.	31
4.3	Reconstruction RMSE (condition C) as a function of anchor weight λ (30 images).	33
4.4	VLA spatial accuracy across the two training iterations. Clean checkpoints only (parse failures ≤ 5).	34
4.5	Grasp-feasibility comparison between training iterations under identical mask-exact evaluation. Wilson 95% confidence intervals in brackets. ON-TARGET rates use the scene-clustered bootstrap interval; ON-OBJECT and per-cell breakdowns retain Wilson intervals.	37
4.6	Run 2 grasp accuracy by sample type. Wilson 95% CIs in brackets.	37
4.7	Run 2 grasp accuracy by object-area tercile. Wilson 95% CIs in brackets.	39

Abstract

Two issues make it difficult to apply modern Vision-Language-Action (VLA) models to recycling. First, transparent plastics cannot be detected by the active infrared depth sensors used in robotic sorting. Second, there is no existing dataset of robot demonstrations for handling transparent waste. This dissertation investigates whether a 7-billion-parameter VLA can learn to sort transparent waste using only synthetic training data. To address the hardware limitations, the project also introduces a software solution to reconstruct the missing depth information.

To achieve this, we developed a cascaded framework comprising a procedural data generator (ReSort-IT), a geometry-aware perception front-end with metric depth reconstruction, and a parameter-efficient Vision-Language-Action policy. Results were evaluated using statistical testing. The perception system achieved a 19.6° mean normal error and a 91% intersection-over-union accuracy on familiar data, and it generalized well to new shapes that looked geometrically similar. Furthermore, an ablation study showed that the depth reconstruction performed just as well as a strong baseline when given perfect surface normals. This means the 0.21 m error we saw came entirely from mistakes in the initial perception step.

On top of this, our first control policy ended up memorizing shortcuts. This issue was hidden by a 97.7% accuracy rate and zero training loss. After fixing the data pipeline, the exact on-target grasp rate nearly tripled to 7.2%. We also found a large gap in performance between using real backgrounds versus composited ones (11.3% compared to 3.1%). This difference suggests the model relies on real-world transparency cues that get lost during the image compositing process.

Two additional tests proved our main finding. Giving the model more varied instructions or using a stronger training setup did not improve its ability to locate objects in space. The model did learn to output the correct text format, going from gibberish to zero errors across eleven tests, but it never actually understood where the objects were. A test on the base model confirmed this lack of physical understanding was caused by the fine-tuning process itself. The system runs at 0.34FPS, which is currently limited by the CPU processing the depth data.

In the end, the model learned a repetitive action pattern instead of true scene understanding. This work explains why training VLAs on transparent objects using only synthetic data fails. The training data is what limits the model, not its architecture or how it's trained.

Keywords: Vision-Language-Action models; transparent object perception; robotic waste sorting; synthetic data generation; depth reconstruction; parameter-efficient fine-tuning.

Chapter 1

Introduction

1.1 Motivation

Automated waste sorting is necessary for recycling to work on a large scale. Robotic systems are already quite good at sorting opaque materials because colour, texture, and reliable depth sensors make it easy to recognize and pick up items [6, 7]. However, transparent plastics are still a major issue. Clear containers like PET bottles make up a huge and valuable part of recyclable waste, but they cannot be detected by the standard infrared depth sensors that most robots use. Because light passes right through or bounces off clear plastic, the sensors end up returning blank or broken depth readings exactly where the robot needs to grasp [2].

At the same time, new AI models for robot control are becoming available. Vision-Language-Action (VLA) models, like RT-2 [5] and OpenVLA [4], connect large language models directly to robot movements. They offer the kind of flexibility that traditional sorting setups lack, as they can follow natural-language instructions and use general internet knowledge to handle new objects they haven't seen before. This is useful for waste sorting since the variety of trash is practically endless. The problem is that VLAs are trained on massive datasets of real robot movements, and there is no existing dataset for handling transparent waste. Furthermore, collecting enough real-world data for this specific task is simply not practical.

1.2 Aim and Objectives

This dissertation aims to design, build, and evaluate a semantic sorting pipeline for transparent plastic waste, driven entirely by generated synthetic data and a depth repair module. As a lab-based feasibility study, the goal is not to deploy a commercial sorter, but to systematically map the performance boundaries of synthetic-only VLA training.

To achieve this aim, the work is structured around five clear objectives:

1. **Synthetic Data Generation:** To build an automated procedural generation pipeline (ReSort-IT) and determine the specific data properties required to teach a VLA spatial targeting without shortcut memorisation.

2. **Geometry Perception:** To train an RGB-only convolutional network to recover the surface geometry of transparent objects and measure its generalisation on unseen shapes.
3. **Depth Reconstruction:** To implement a normal-guided depth repair module, compare it against baseline image-infilling, and quantify exactly how perception errors propagate into metric depth.
4. **Computational Profiling:** To profile the end-to-end latency of the integrated pipeline and identify what is slowing it down.
5. **VLA Semantic Grounding:** To fine-tune a Vision-Language-Action model (Open-VLA) and evaluate whether its spatial targeting is genuinely grounded in the text instruction, testing its robustness against varied phrasing.

1.3 Contributions

The principal contributions of this dissertation are:

- **The ReSort-IT Pipeline:** A procedural generation tool that converts 3D transparent object models into image–language–action training triplets.
- **Error Propagation Analysis:** A controlled ablation study quantifying exactly how surface-normal prediction errors degrade downstream metric depth.
- **Format vs. Grounding Diagnosis:** A fine-tuning evaluation proving that VLA output formatting and semantic spatial grounding are independent properties. This establishes that the training recipe controls the formatting of the output. On the other hand, the model’s spatial grounding depends entirely on how realistic the data is.
- **End-to-End System Evaluation:** A latency and grasp-feasibility breakdown of a synthetic-only VLA pipeline, identifying how realistic the training data is, and the CPU-bound depth solver, as the two main reasons for performance limitations.

1.4 Dissertation Structure

Chapter 2 reviews the literature on robotic waste sorting, transparent-object perception, and VLA fine-tuning, establishing the research gap this project targets. Chapter 3 details the methodology, including the ReSort-IT data generator, the perception network, the depth solver, and the fine-tuning setup. Chapter 4 presents the quantitative results for both the individual components and the complete integrated system. Chapter 5 discusses these findings in the context of the research questions and the wider literature. Finally, Chapter 6 concludes the study, outlines its limitations, and suggests directions for future work.

Chapter 2

Literature Review

2.1 Introduction

Plastic waste sorting faces two main challenges: a global recycling shortfall and a bottleneck in robotic automation. Less than 10% of plastic waste is recycled globally, often due to inadequate sorting infrastructure [8]. Industrial Material Recovery Facilities (MRFs) rely majorly on near-infrared (NIR) optical sensors and rigid, rule-based controllers. These systems struggle when waste materials vary, deform, or are contaminated [7, 9]. Vision-Language-Action (VLA) models provide an alternative approach. By combining visual perception, language instructions, and action generation into a single architecture trained on internet-scale data, they can handle generalisation across different real waste morphologies without needing hard-coded rules for every plastic category [10, 11].

Applying these models to actual sorting lines, however, introduces several problems. This dissertation addresses two specific issues. First, transparent plastics cannot be detected by standard depth sensors. Second, there is no existing dataset of robot demonstrations to fine-tune a VLA for this task.

This chapter maps out the existing research to contextualize these challenges. Section 2.2 reviews classical sorting methods. Section 2.3 covers scene understanding and transparent-object perception. Sections 2.4–2.5 explain the VLA framework, its spatial limitations, and how these models are fine-tuned. Section 2.6 details synthetic data strategies and their known risks. Finally, Section 2.7 summarizes the technical gaps, leading into the main research aim and questions in Section 2.8.

2.2 Classical Plastic Sorting: Sensing and Classification

Lubongo et al. [7] survey current MRF sorting technology and document three broad approaches: mechanical separation by density, electromagnetic extraction of ferrous materials, and optical sorting using spectral sensors. Among these, NIR spectroscopy is the industrial standard. It permits contact-free identification of polymer types based on their molecular absorption spectra. Under controlled conditions, commercial NIR systems achieve throughputs of 60 to 80 picks per minute with classification accuracies above 99% [7].

However, these classical systems face hardware limitations. Standard NIR equipment cannot identify black plastics because the carbon-black pigment absorbs nearly all incident light across the $0.9\text{--}1.7\,\mu\text{m}$ range, rendering these items invisible to the sorter [9]. Mid-wave infrared hyperspectral imaging ($3\text{--}5\,\mu\text{m}$), where plastics exhibit distinct absorption features regardless of colour, provides an effective solution, though at a considerably higher hardware and processing cost [9]. Contamination also degrades accuracy. Food residue and labels interfere with both NIR and visual sensors [8, 9], and rigid rule-based systems lack the contextual reasoning required to recognise a contaminated PET bottle as PET.

On the machine-vision side, RGB-based classification has progressed steadily. Bobulski and Kubanek [12] benchmark CNNs for polymer classification using low-cost RGB cameras, reporting a 74% cross-validated accuracy for a lightweight 15-layer network and 86% for MobileNet-v2. This is sufficient for broad material categories, but not for fine-grained polymer discrimination. One-stage detectors such as YOLO variants have become the strongest choice for real-time conveyor detection [13], and edge-deployable classifiers reach high accuracy on curated waste imagery [14]. Kroell et al. [9] conclude that deep learning has improved classification flexibility. Yet, distinguishing chemically distinct but visually identical polymers, such as clear PET from clear PVC, remains unsolved by RGB methods alone. Spectral data remains a necessity wherever strict chemical accuracy is required [7, 15].

The training data used for these deployed commercial systems relies on massive datasets of real conveyor imagery. These datasets capture items that are crushed, overlapping, soiled, and degraded, not clean laboratory samples. Leading industrial operators treat these proprietary real-waste datasets as their core asset [7]. The public research equivalent is ZeroWaste [1], the first industrial-grade waste detection and segmentation dataset. Its authors note that recyclable-waste perception requires detecting highly deformable and often translucent objects in extreme clutter, lacking the contextual clues found in human-centric datasets. This reality highlights the main data boundary of this dissertation. The ClearGrasp assets used here depict clean, pristine, isolated transparent objects, representing the opposite extreme of the material used to train deployed systems. Consequently, ZeroWaste is used in this work not as training data, but as the real-world benchmark to measure transfer performance.

Finally, spectroscopic and RGB classification systems answer what is on the belt, but they do not determine where to grasp it. Converting a classification into a successful physical pick on cluttered, deformed waste is often identified as a harder problem than the classification step itself [7]. Koskinopoulou et al. [6] present an integrated vision-based robotic sorting system on an industrial line that combines deep learning categorisation, grasp selection, and robotic separation. They report a sorting accuracy above 90%. This system shows current industry standards as well as their constraints. The perception stack assumes the availability of usable depth sensing. Moreover, extending the system to handle new categories or follow new sorting instructions requires full retraining and re-engineering, rather than simply issuing a new text command.

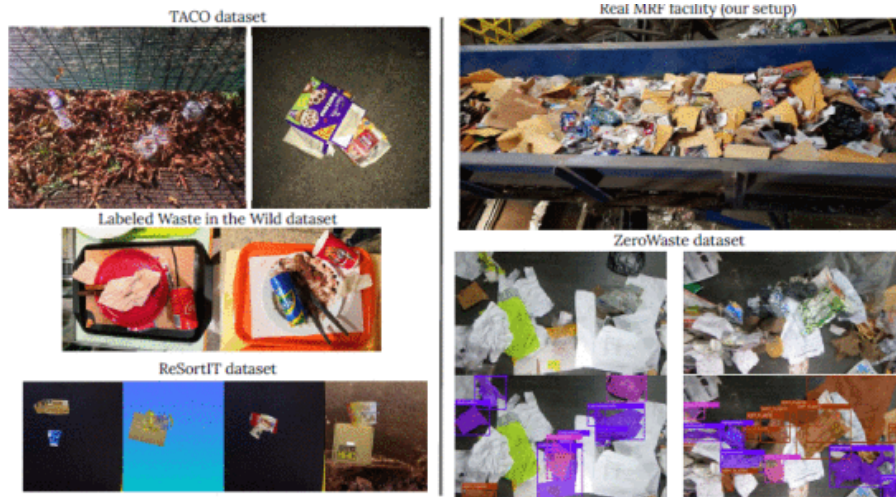


Figure 2.1: An example frame from the ZeroWaste dataset illustrating the extreme clutter, deformation, and contamination typical of real-world material recovery facilities. This represents the domain gap that synthetic-only VLA models must eventually cross. Figure reproduced from Bashkirova et al. [1].

2.3 Semantic Scene Understanding for Robotic Sorting

Transforming raw sensor data into actionable formats for sorting relies on three components: open-vocabulary detection, depth reconstruction, and affordance prediction.

Traditional detectors require a fixed list of categories defined during training, making them fail on novel packaging. VoxPoser [16] reveals a more flexible alternative by linking an open-vocabulary detector (OWL-ViT) with the Segment Anything Model. This combination identifies and masks arbitrary objects described in natural language (such as “the crushed translucent bottle”) without needing labelled examples. This capability enables language-conditioned sorting. However, VoxPoser’s downstream system builds 3D value maps over voxel grids, meaning it assumes the 3D perception is reliable. Transparent objects break this assumption.

Depth reconstruction for transparent objects is the second component. Active infrared sensors, like the Intel RealSense series, project structured light that either refracts through or passes across clear PET and PVC. This creates depth maps with severe holes and distortions exactly where the robot needs to grasp [2]. ClearGrasp [2] solves this problem directly (Figure 2.2). Using standard RGB images, it predicts surface normals, transparent-object masks, and occlusion boundaries through three parallel DeepLabV3 networks. It then runs a global optimisation to calculate the missing depth over the corrupted areas, using the reliable depth data from the surrounding opaque scene as an anchor. ClearGrasp was trained on a physically based synthetic dataset of over 50,000 images (which also provides the 3D assets used in this dissertation). By applying this RGB-to-geometry approach, the authors increased the success rate of parallel-jaw grasps on real transparent objects from a 12% baseline to 74%.

This project adopts two key elements from ClearGrasp: the use of synthetic training data, and the two-step process of predicting geometry from RGB before running an optimi-

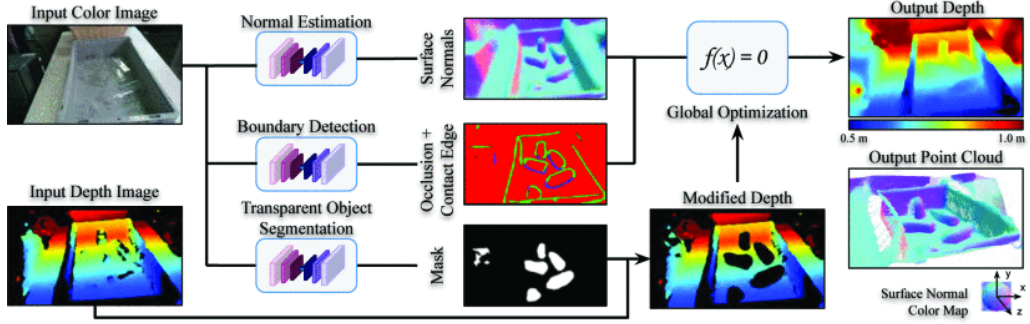


Figure 2.2: The ClearGrasp depth reconstruction pipeline. Three parallel networks infer surface normals, transparent-object masks, and occlusion boundaries from an RGB-D input. A global optimisation step then uses these outputs to correct and complete the raw depth map. Figure reproduced from Sajjan et al. [2].

sation. We do not use their final step, because ClearGrasp ends with a task-specific grasp network that lacks a semantic or language interface. Additionally, ClearGrasp’s training renders are physically based, meaning the refraction, caustics, and shadows visually match the scene’s lighting. Chapter 4 confirms that preserving this visual consistency in the generated data shapes the policy’s final performance.

Affordance prediction connects perception to action. VoxPoser uses LLM-generated value maps to assign rewards to actionable areas, like a bottle neck [16]. Alternatively, RoboPoint [3] (shown in Figure 2.3) trains a vision-language model to directly predict 2D coordinates for grasp targets. This trades 3D resolution for faster inference speed, and it is structurally similar to the coordinate-token prediction evaluated in this work. SpatialVLA [17] takes this further by adding egocentric 3D representations into the VLA action-token space, which helps the system better understand spatial relationships in cluttered environments.

2.4 Vision-Language-Action Models

The VLA framework changes how robotic systems are built. Instead of combining separate modules for perception, planning, and control, a single model learns to map visual inputs and text commands directly to actions [10, 11]. Recent surveys identify over eighty VLA models published since 2022. These are generally categorised by how they generate actions (such as autoregressive token prediction, diffusion, and flow matching) and by their function. These functions include sensorimotor models, world models, and affordance-based models. Affordance-based models are useful for unstructured environments because the action they generate is not tied to the physical constraints of a specific robot [11].

RT-2 [5] scaled up the token prediction approach. As illustrated in Figure 2.5, it converts continuous robot movements and gripper commands into 256 discrete text bins. By fine-tuning the model on internet-scale vision-language data alongside robot trajectories, RT-2 learns physical skills without forgetting its broad semantic knowledge. This allows RT-2 to manipulate objects missing from the robot dataset simply by relying on general web knowledge, which is highly valuable for sorting unpredictable waste streams. How-

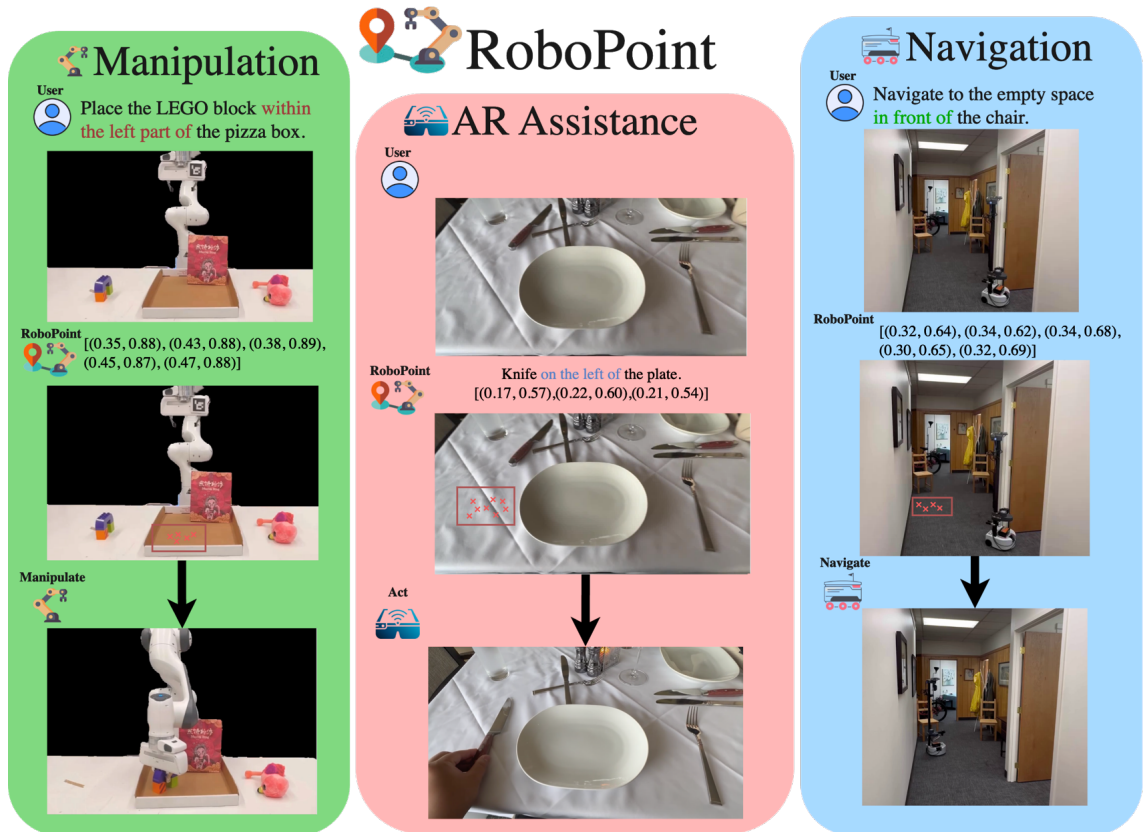


Figure 2.3: Spatial affordance prediction using RoboPoint. Rather than outputting bounding boxes, the vision-language model translates natural language instructions into precise 2D spatial coordinates (affordances) for robotic manipulation and navigation. Figure reproduced from Yuan et al. [3].

ever, RT-2 is closed-source and computationally expensive. Its 55-billion-parameter version runs at just 1–3 Hz on multi-TPU cloud hardware, and the smaller 5-billion version runs at approximately 5 Hz [5].

OpenVLA [4] made this technology more accessible. As illustrated in Figure 2.4, it is a 7-billion-parameter open-source model that pairs a fused SigLIP and DINOv2 vision encoder with a Llama-2 backbone. It was trained on 970,000 real-robot episodes and uses the same 256-bin action tokenisation as RT-2. For resource-limited projects, Kim et al. [4] showed that OpenVLA can be adapted using Low-Rank Adaptation [18] with just tens of demonstrations. Adding 4-bit quantisation [19] makes it possible to fine-tune the model on a single high-memory GPU, which is the exact hardware setup used in Chapter 3.

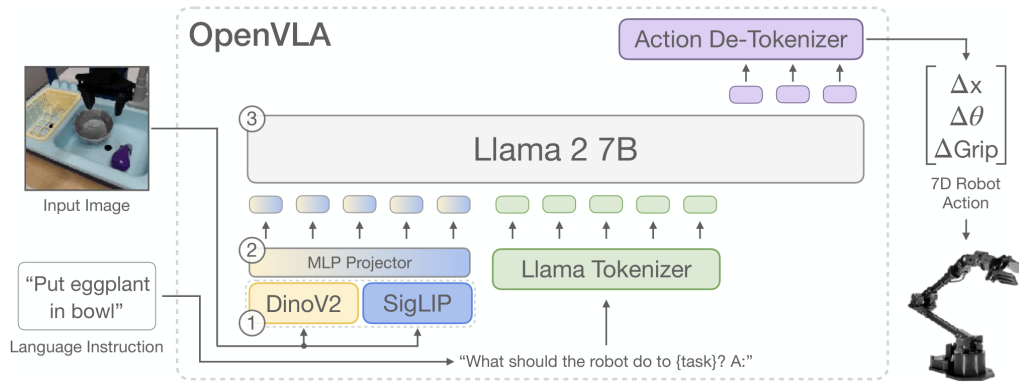


Figure 2.4: The OpenVLA model architecture. A vision encoder (fusing DINOv2 and SigLIP features) and a text instruction are mapped into a 7-billion-parameter Llama-2 backbone, which outputs 7-dimensional robot control actions. Figure reproduced from Kim et al. [4].

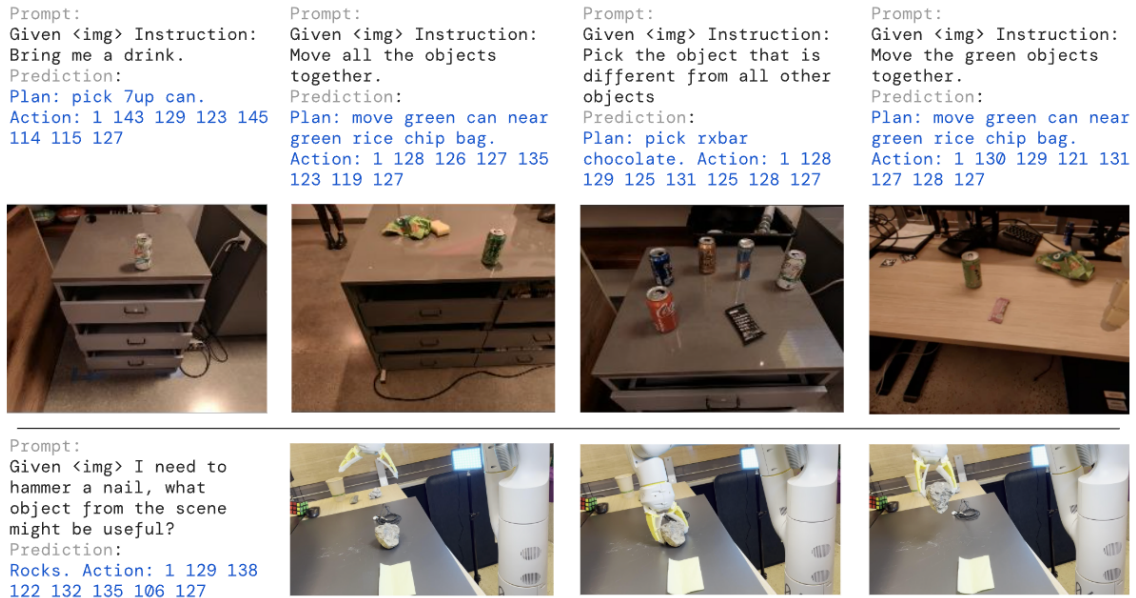


Figure 2.5: The action tokenisation scheme introduced by RT-2 and adopted by OpenVLA. The model predicts a high-level text plan followed by a sequence of discrete text tokens (numbers) representing the continuous 7-dimensional robot control actions. Figure reproduced from Zitkovich et al. [5].

The choice of how to tokenise actions remains an active research area [20]. Uniform binning is the simplest method, but its 256-level granularity sets a hard limit on spatial precision. Additionally, as explored in Chapter 4, outputting actions as text links the validity of the motor command to the stability of the language model itself. This creates a potential point of failure that continuous-action architectures naturally avoid. Other recent models build on this foundation by adding intermediate reasoning steps (such as visual chain-of-thought in CoT-VLA [21]), explicit 3D world models [22], or extra physical sensors (OmniVLA [23]).

2.4.1 Spatial Grounding: A Documented Weakness

Recent evaluations show that VLAs and their underlying vision-language models have strong semantic understanding but weak spatial precision. SpatialBench [24] documents systematic deficits in localisation and spatial-relation reasoning, even when the model’s object recognition is nearly perfect. The affordance and representation methods discussed in Section 2.3 (such as RoboPoint’s dedicated pointing supervision [3], SpatialVLA’s 3D encodings [17], and 3D-VLA’s generative world model [22]) are direct responses to this weakness. Spatial grounding does not naturally emerge just by making models larger; it must be engineered through specific data, representations, or architectures. For robotic sorting, this means recognising a bottle is not enough. The system must output coordinates accurate to a gripper’s mechanical tolerance. This dissertation tests this limit by measuring how well computer-generated 2D images can teach spatial grounding to a standard OpenVLA model, and it documents exactly what happens when that data falls short.

2.5 Fine-Tuning Practice: Recipes, Forgetting, and What Adaptation Preserves

Because the results of this project depend on how OpenVLA was adapted, the fine-tuning literature requires direct review.

Adapter placement and strength. The original LoRA setup only adapted the attention query and value projections [18], and this minimal configuration remains widely used. However, recent evidence pushes beyond it. The QLoRA study showed that applying low-rank adapters to all linear projections (both attention and MLP) is necessary to match the quality of full fine-tuning. Once all linear layers are targeted, the specific adapter rank matters little [19]. The optimised fine-tuning recipe published for OpenVLA (OpenVLA-OFT) follows this exact principle. It applies LoRA to all linear layers of the backbone. It also replaces autoregressive token prediction with parallel decoding over a continuous action head trained by L1 regression. This upgrade raised LIBERO success from 76.5% to 97.1% while accelerating generation [25]. OFT’s extended variant additionally injects FiLM conditioning to improve language grounding [25]. Run 4 in Chapter 3 directly tests this literature. It upgrades a basic attention-only baseline to an all-linear, higher-rate configuration to test whether earlier failures stemmed from a weak training setup.

What fine-tuning destroys. A parallel line of research shows that narrow fine-tuning can degrade a base model’s existing capabilities. Catastrophic forgetting is common in LLMs between 1B and 7B parameters during continual instruction tuning. This forgetting affects domain knowledge, reasoning, and comprehension. Surprisingly, this degradation actually worsens as model size increases within that range [26]. Biderman et al. [27] clarify this for parameter-efficient methods, showing that LoRA learns less and forgets less than full fine-tuning. It acts as a regulariser that preserves out-of-domain behaviour at the cost of some in-domain performance gains. This has two main implications for this project. First, previous studies measure forgetting against standard benchmarks. How a narrow robotics fine-tune affects a specific functional pathway, such as a VLA’s instruction conditioning, is less clear. The base-model control in Chapter 4 is designed to measure exactly that. Second, methods like RT-2’s web-data co-fine-tuning [5] and OFT’s FiLM conditioning [25] act as defences against this degradation by keeping the language channel active during adaptation. The training data used in this project determines targets purely based on the image, offering no such defence. Chapter 4 measures the consequences of this design choice.

2.6 Synthetic Data and Its Hazards

Collecting real robot demonstrations is expensive, and for transparent waste, no public dataset exists. Two main synthetic strategies exist: rendering and compositing.

Rendering versus compositing. Physically based rendering, used in ClearGrasp [2] and in recent industrial quality-control pipelines built in Blender [28], produces high-fidelity images with exact ground truth. However, it requires complex 3D assets and significant rendering infrastructure. Compositing involves cutting segmented foregrounds and pasting them onto various backgrounds. It is significantly cheaper and well-established. Dwibedi et al. [29] showed that automatically cut-and-pasted objects can train competitive detectors, provided the system actively suppresses pixel artefacts like sharp boundaries. Otherwise, the model simply learns to detect the artefacts rather than the objects. Later work proved that placement context also matters. Randomizing pose, scale, lighting, and backgrounds is the standard way to force the model to rely on object-intrinsic cues rather than background clues [30].

Why standard compositing fails for transparent objects. The cut-and-paste method assumes that patch-level realism is enough [29]. This works for opaque objects because their appearance is mostly intrinsic. Transparent objects violate this assumption entirely. Their appearance depends heavily on the background seen through them via refraction, absorption, and caustics, along with physically consistent shadows and specular highlights. Therefore, pasting a transparent object onto an arbitrary background does more than just create an unrealistic seam. It breaks the physical link between the object’s position and its appearance, removing the exact visual cues needed to locate it. Combining the literature on compositing and transparent perception predicts this exact failure, yet it remains largely

untested. Compositing studies evaluate opaque objects, and transparent-object research uses physically based rendering. The native-versus-composite results in Chapter 4 provide a measured data point at this intersection.

Hazards specific to action supervision. There is also a specific risk when using composited data to train action outputs. If the target labels correlate with the background, or if pasting artefacts are obvious, a large model can minimize its training loss by learning shortcuts that involve zero spatial understanding. Because most parts of an action string are constant, overall token accuracy metrics can easily hide this failure. When the model outputs a class label, shortcut learning is obvious in the accuracy score. When it outputs spatial coordinates buried in text, the failure is harder to spot. Chapter 4 documents a clear example of this failure and introduces a testing protocol to catch it. We show that when the supervised quantity is a spatial action, evaluating per-coordinate error on held-out data and treating zero training loss as a warning sign are critical steps.

The implementation gap. Finally, the risks mentioned above assume the compositing is done perfectly. A real system must deal with imperfect compositing. Random placement and scaling can cause targets to blend into busy backgrounds, shrink objects below the camera’s resolution limit, or degrade faint transparency cues when image noise is added. Chapter 3 shows exactly where the data generator suffers from these issues. Chapter 4 reveals their total cost by measuring the persistent performance gap between composited and native training frames. The composited data used in this project has structural limits when dealing with transparent objects, and properly measuring the impact of those limits is a core finding of this dissertation.

2.7 Technical Gaps for VLA-Based Plastic Sorting

Based on this literature review, four main issues prevent current VLA systems from being deployed in plastic sorting.

Inference latency. Autoregressive VLAs generate actions token by token, which means they cannot reach the speeds required for reactive robot control. RT-2 runs at 1 to 5 Hz depending on its size [5], and OpenVLA runs at roughly 5 Hz on a high-end GPU [4]. Kawaharazuka et al. [11] identify this latency as the main barrier to real-world use. New methods like parallel decoding [25] attempt to fix this issue. However, sorting lines make the problem even harder because the targets are constantly moving.

The embodiment and domain gap. VLA training datasets consist of clean, well-lit household tasks performed by specific robot arms. Industrial waste is deformed, contaminated, cluttered, and moving [1]. This makes it completely different from the training data, both visually and physically [11]. Classical systems like the one developed by Koskinopoulou et al. [6] remain the benchmark for real waste precisely because they were engineered specifically for that environment.

Data scarcity for transparent waste. No demonstration dataset exists for handling transparent waste. Besides, transparent objects break the depth sensors that standard

RGB-D policies and 3D affordance methods rely on (Section 2.3). Synthetic data generation is the only viable alternative. However, as discussed in Section 2.6, this approach has known flaws. For example, compositing images severs the physical link between an object’s position and its appearance. It also introduces a high risk of models learning hidden shortcuts instead of genuine spatial understanding.

Operating without spectral sensors. Current VLAs operate on RGB or RGB-D inputs, meaning they cannot chemically tell the difference between visually identical plastics like clear PET and clear PVC [7]. OmniVLA introduces multi-sensor inputs [23], but this has not yet been applied to NIR polymer classification. This dissertation operates deliberately within this limitation. The transparent target categories are drawn from the ClearGrasp object set and are distinguished by their shape. Chemical identification is out of scope and left for future work.

Table 2.1 summarises the principal approaches against these gaps.

Table 2.1: Summary of key approaches in automated plastic sorting and robotic manipulation.

Study	Modality	Architecture	Key contribution and limitation
Lubongo et al. [7]	NIR spectroscopy	Rule-based	Industrial standard, $\sim 99\%$ accuracy under controlled conditions; blind to black plastics; no grasp geometry.
Kroell et al. [9]	RGB vision	CNN / YOLO	Flexible visual classification; cannot separate chemically distinct, visually identical polymers (PET vs. PVC).
Koskinopoulou et al. [6]	RGB(-D)	Deep classical pipeline	Integrated robotic sorting on an industrial line; presumes valid depth; retraining required per category.
Bashkirova et al. [1]	RGB	Detection/segmentation dataset	First in-the-wild industrial waste dataset (deformable, translucent, cluttered); no manipulation labels.
Sajjan et al. [2]	RGB-D	DeepLabV3 + optimisation	Transparent depth reconstruction (grasp success $12\% \rightarrow 74\%$); no semantic or language interface.
Zitkovich et al. [5]	RGB + text	VLA (co-fine-tuned)	Web-knowledge transfer to control; closed; 1–5 Hz inference.
Kim et al. [4]	RGB + text	Prismatic VLM + Llama-2	Open-source VLA; LoRA-adaptable on one GPU; inherits latency and spatial-precision limits.
Kim et al. [25]	RGB + text	OpenVLA + OFT recipe	All-linear LoRA, parallel decoding, continuous actions; recipe-level fix for latency and action fragility.
Guo et al. [23]	Multi-sensor	Sensor-fused VLA	Non-RGB modalities in VLA representations; unvalidated for NIR sorting.

2.8 Research Gap, Aim, and Questions

No prior work combines the specific elements assembled in this review. This project introduces a VLA fine-tuned for grasping transparent waste using exclusively computer-generated data. It evaluates this model alongside a software front-end designed to repair broken depth data, tracking how errors in the perception step propagate to the final grasp target. Additionally, it diagnoses the risk of models learning hidden shortcuts from synthetic action data and treats the adaptation recipe as an experimental variable.

Working within the hardware constraints of this project, the study is narrowed to answer the core technical question on which deployment depends. Under what data conditions does synthetic-only training actually teach a VLA spatial grounding on transparent objects, and how do the errors from the perception and policy stages compound?

The aim of this work is to design, build, and evaluate a software pipeline that predicts where to grasp transparent plastic waste based on text instructions, trained entirely on synthetic data. This aim is designed to answer the following research questions:

- RQ1** Can procedurally generated synthetic data alone teach a VLA to spatially locate transparent objects, and what specific data properties dictate success or failure?
- RQ2** How accurately can an RGB-only convolutional network recover the surface geometry of transparent objects, and how does this accuracy degrade on unseen shapes?
- RQ3** Does normal-guided depth reconstruction outperform baseline image-infilling, and how do perception errors propagate into metric depth?
- RQ4** Is the integrated pipeline computationally feasible for real-time conveyor operation, and what is the primary latency bottleneck?
- RQ5** Does the fine-tuned model condition its spatial targeting on the language instruction, and does it remain robust when instructions are phrased differently?

The following chapter details the methodology used to answer these questions.

Chapter 3

Methodology

This chapter describes the design and implementation of a software pipeline for the semantic sorting of transparent plastic waste. To overcome the sensor and data limitations outlined in Chapter 1, the pipeline is organised as a cascade of three work packages as illustrated in Figure 3.1.

Work Package 1 (WP1) is a procedural data generation framework, called ReSort-IT. It generates image, language, and action training triplets from existing transparent-object assets without requiring physical robot demonstrations. Work Package 2 (WP2) is a multi-task convolutional network that predicts surface normals and segmentation masks from RGB input, feeding a depth reconstruction solver that repairs the corrupted sensor stream. Work Package 3 (WP3) fine-tunes a pre-trained 7-billion-parameter Vision-Language-Action model [4] on the synthetic data when updating only a fraction of the parameters. The chapter closes with the evaluation methodology used to assess each component and the integrated system.

A cascaded design was chosen over end-to-end training for three reasons. First, it decouples metric depth recovery from semantic reasoning, so each component can be evaluated, debugged, and replaced independently. Second, it keeps the large language-model backbone isolated from geometric computation, for which it is poorly suited. Third, it permits the perception front-end and the policy to be trained on different data sources. This is necessary because ground-truth geometry (normals and depth) and action labels come from different generation processes.

3.1 Work Package 1: Synthetic Data Generation (ReSort-IT)

3.1.1 Source Assets

Foreground objects were sourced from the ClearGrasp synthetic dataset [2], which provides photorealistic renders of transparent plastic objects with per-pixel segmentation masks, surface normals, and depth (visualised in Figure 3.3). The **square-plastic-bottle** category, comprising 11,702 RGB and mask pairs, was used to train the manipulation policy. Background plates were drawn from a separately collected pool of 153 texture images spanning industrial and domestic surfaces (including conveyor rubber, wood, concrete, fabrics,

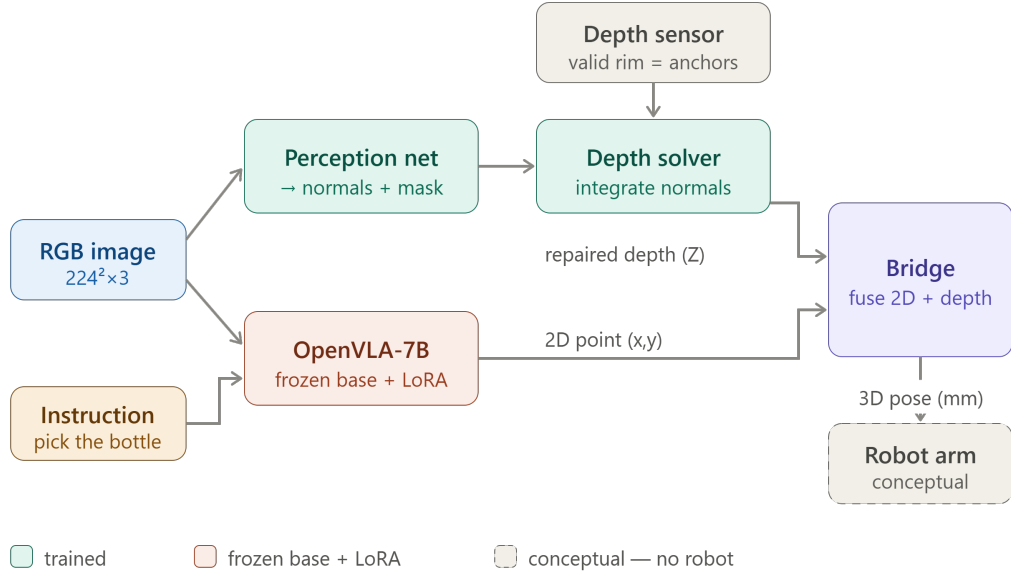


Figure 3.1: End-to-end system architecture. The WP2 perception module provides repaired metric depth, which combines with the WP3 VLA policy’s 2D spatial tokens to yield 3D workspace coordinates via kinematics.

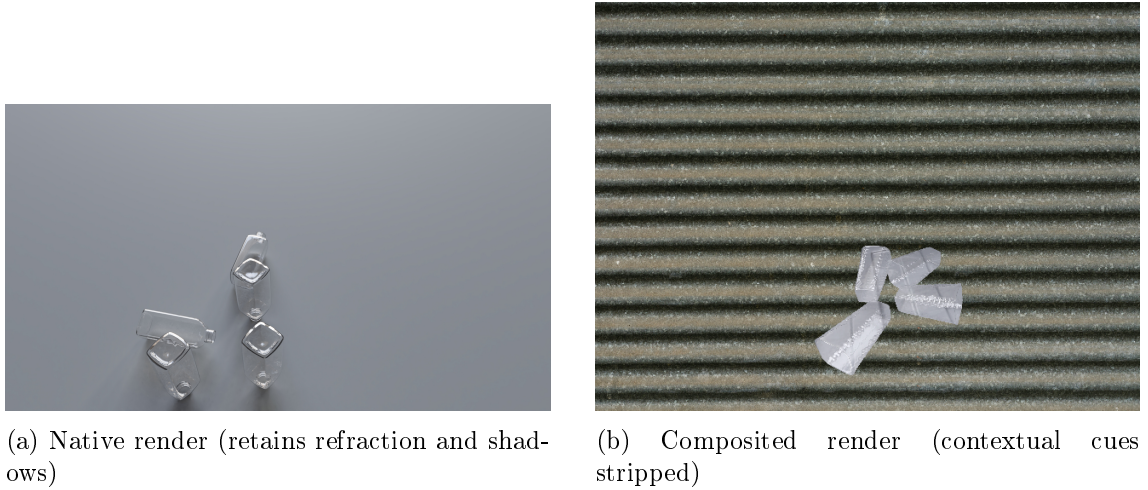


Figure 3.2: Visual comparison of a native scene versus procedurally composited over an arbitrary background. The compositing process degrades the environmental lighting interactions (refraction, caustics, and cast shadows) that serve as critical grounding cues for the VLA.

and patterned materials).

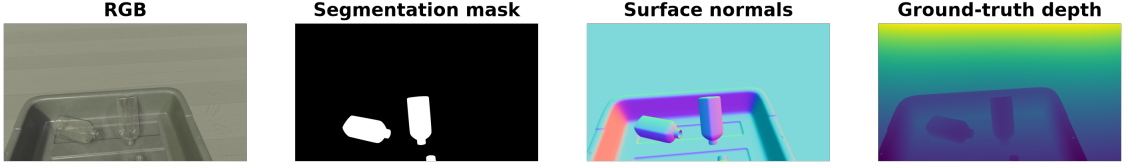


Figure 3.3: Intermediate representations provided by the ClearGrasp source assets. From left to right: the RGB image, the binary segmentation mask, the ground-truth surface normals, and the depth map. These isolated modalities form the foundation of the synthetic training pipeline.

Policy training was restricted to a single object category. This separates the core research question (whether a VLA can learn spatial grounding of transparent objects from composited synthetic data) from the broader challenge of multi-class generalisation. The remaining ClearGrasp categories were reserved as held-out data to evaluate how well the perception module generalises to new shapes (Section 3.4).

3.1.2 Grasp Target Derivation

Each ClearGrasp render may contain several disconnected transparent objects. Calculating a grasp target from the centroid of the full mask fails in this situation because averaging over multiple objects places the target in the empty space between them. We observed this failure during early development and refer to it as the *empty-centre* problem. On a physical sorting line, this would direct the robot to grip the bare conveyor surface.

To resolve this, the pipeline first separates individual objects using connected-component analysis. The binary mask $\mathbf{M}$ is decomposed into external contours,

$$\mathcal{C} = \{c_1, \dots, c_n\} = \text{findContours}(\mathbf{M}), \quad (3.1)$$

and one contour c_i is selected uniformly at random as the grasp target for that composite. The remaining objects are kept in the frame as clutter. The target coordinate is then the centre of mass of the selected contour, computed from its image moments,

$$M_{pq} = \sum_{(x,y) \in c_i} x^p y^q, \quad c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}}, \quad (3.2)$$

which places the ground-truth target on solid plastic rather than in the gap between objects as illustrated in Figure 3.4. However, this calculation fails for concave objects. For crescent-shaped or crushed items, the centre of mass can fall outside the object. An audit of the first generated dataset (Section 4.4.1) showed that 10% of targets suffered from this defect.

Consequently, the final generator validates every centroid with a point-in-polygon test,

$$\text{pointPolygonTest}(c_i, (c_x, c_y)) \geq 0, \quad (3.3)$$

and if the test fails, it substitutes the contour’s most-interior point (the argmax of the L2

distance transform of the filled contour). By mathematical definition, this point always lies on the object material. Additionally, a runtime assertion verifies every generated label against its saved object mask, guaranteeing 100% label validity in the final dataset.

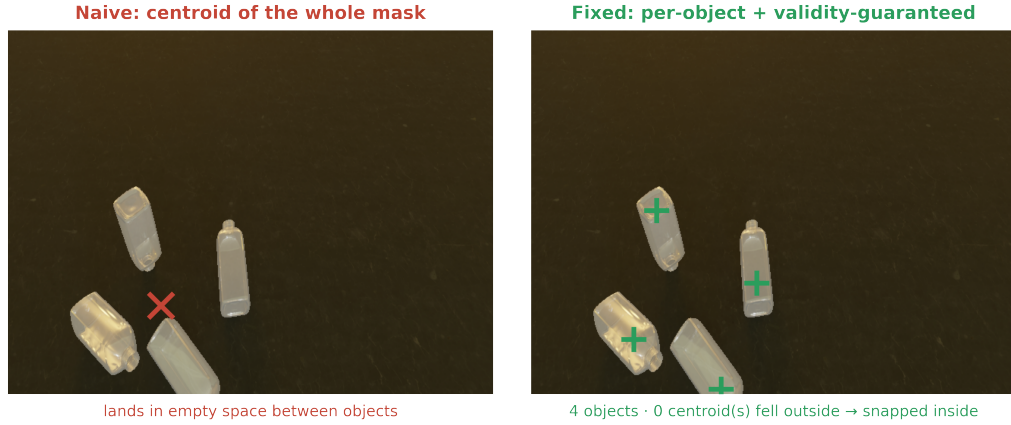


Figure 3.4: Deriving where to grasp complex objects. (Left) The object segmentation mask. (Middle) The naive geometric centroid falls outside the physical object boundary due to the concave shape, producing an invalid grasp target. (Right) The corrected target, derived by calculating the distance transform of the mask and extracting the internal point furthest from all boundaries.

3.1.3 Compositing and Domain Randomisation

Each source image is composited onto randomly selected backgrounds three times, with independently sampled transformations. This yields a total of 35,103 composites for the initial baseline dataset (Run 1). The following randomisations are applied to every composite:

- **Scale.** The foreground is resized so that its width is a fraction $s \sim \mathcal{U}(0.25, 0.75)$ of the 640-pixel background width. This simulates both distant and close object appearances.
- **Geometric transforms.** A horizontal flip is applied with a probability of 0.5, followed by a rotation of $\theta \sim \mathcal{U}(-15^\circ, 15^\circ)$. The mask is re-thresholded after rotation to remove interpolation artefacts.
- **Photometric jitter.** Foreground contrast is scaled by $\alpha \sim \mathcal{U}(0.6, 1.4)$ and brightness shifted by $\beta \sim \mathcal{U}(-40, 40)$. In HSV space, the hue is shifted by up to $\pm 10^\circ$ and saturation is scaled by $\mathcal{U}(0.7, 1.3)$.
- **Edge softening.** The binary mask is blurred with a 7×7 Gaussian kernel before alpha blending. This removes the sharp compositing seam at the object boundary, preventing the network from learning a localisation shortcut that is unrelated to the object’s actual appearance.
- **Sensor noise.** Zero-mean Gaussian noise with $\sigma \sim \mathcal{U}(1, 5)$ intensity levels is added to the final composite.

- **Position.** The composite location is sampled uniformly, ensuring the object remains fully in the frame. This placement is independent of the background identity, ensuring the model cannot use the background appearance to guess the target position.

Alpha blending with the softened mask $\tilde{\mathbf{M}}$ produces the final composite:

$$\mathbf{I}_{\text{comp}} = \mathbf{I}_{\text{fg}} \odot \frac{\tilde{\mathbf{M}}}{255} + \mathbf{I}_{\text{bg}} \odot \left(1 - \frac{\tilde{\mathbf{M}}}{255}\right). \quad (3.4)$$

3.1.4 Real-Frame Mixing and Native Resolution

The final dataset corrects two properties of the first-generation dataset that caused the spatial-precision ceiling measured in Chapter 4.

First, alpha compositing removes the exact visual cues needed to locate transparent objects in real images, such as background refraction, cast and contact shadows, and physically consistent specular highlights (compared side-by-side in Figure 3.2). A pasted transparent object looks identical regardless of what lies behind it. As a result, no training signal links the object’s position to its transparency-specific appearance. To fix this, the final dataset mixes two sources in roughly equal proportion. Each source image contributes two composites and two lightly augmented native ClearGrasp frames. These native frames only receive horizontal flips, photometric jitter, and sensor noise. Rotation is excluded to preserve the authentic render geometry. The frames are labelled from their own segmentation masks using the target derivation method described in Section 3.1.2, which ensures all coordinates are valid. The native frames provide true refraction and shadowing; the composites provide background diversity.

Second, the first-generation samples were rendered at 640×480 and subsequently resized to the policy’s 224×224 encoder input, compounding two lossy resamplings. The final dataset is generated directly at 224×224 . This ensures objects are downsampled only once under controlled interpolation. At this resolution, one action bin corresponds to 0.875 pixels, meaning tokenisation granularity is no longer a limiting factor on achievable precision. The resulting split (Table 3.1) contains nominally equal numbers of composites and native frames.

Table 3.1: Final (Run 2) dataset composition by split and sample type.^a

Split	Composites	Native frames	Total
Train	14,470	16,380	30,850
Validation	3,110	3,510	6,620
Test	3,095	3,512	6,607
Total	20,675	23,402	44,077

^aBoth sample types begin with the same number of generation attempts per source frame. The slight imbalance ($\approx 53\%$ native) occurs because composited samples are rejected more often by the target-validity filters (Section 3.1.2). Random placement and geometric transforms frequently push the target off-frame, shrink it below the minimum-area threshold, or break mask overlap. Native frames are static and pass these checks at a higher rate.

3.1.5 Data Partitioning

Data leakage between splits was controlled at two levels. First, source images were partitioned 70/15/15 into train, validation, and test sets before compositing. This ensures no rendered object appears in more than one split. Second, the 153 background plates were partitioned using the same ratios (107/22/24), and each split draws backgrounds exclusively from its own pool. Without this second control, the model could artificially lower its validation error simply by memorising a background seen during training.

The resulting initial dataset (Run 1) contains 24,570 training, 5,265 validation, and 5,268 test composites (35,103 in total). The composition of the final dataset (Run 2), which incorporates native frames, is detailed in Section 3.1.4 and Table 3.1.

3.1.6 Action Representation and Tokenisation

Each composite is paired with a natural-language instruction (such as “Pick up the transparent plastic bottle.”) and a 7-dimensional action vector. This follows the OpenVLA convention, which includes planar target coordinates (x, y) , height z , orientation (roll, pitch, yaw), and gripper state. The target coordinates are normalised by the image dimensions. The remaining dimensions are fixed at neutral values ($z = \text{roll} = \text{pitch} = \text{yaw} = 0.5$, gripper = 1.0). This setup reflects the top-down grasp geometry of a conveyor workspace where only the planar target changes between samples.

Because the language model operates on discrete tokens, each continuous dimension $a \in [0, 1]$ is quantised into 256 uniform bins,

$$a_{\text{quant}} = \text{clip}(\text{round}(255 a), 0, 255), \quad (3.5)$$

and mapped to a reserved vocabulary token $\langle \text{action_}k \rangle$, $k \in \{0, \dots, 255\}$.

As noted in Section 3.1.4, one bin corresponds to 0.875 pixels along each axis at the 224×224 input resolution. The complete training sample pairs the composite image with the following prompt:

```
In:  What action should the robot take to {instruction}\n Out:  <action_x>
      <action_y> ...<action_g>
```

The `\n` in the prompt denotes an actual newline character. A bug in an early evaluation script (discussed in Chapter 4) mistakenly printed the literal two-character sequence, but this error was never present in the actual training data.

3.2 Work Package 2: Perception Front-End

3.2.1 Network Architecture

The perception module (detailed in Figure 3.5) is a dual-head, fully convolutional network built on DeepLabV3 [31] using an ImageNet-pretrained ResNet-50 backbone. The backbone and the Atrous Spatial Pyramid Pooling stage are shared. The final classifier is

replaced by two parallel 1×1 convolutional heads that operate on the shared 256-channel feature map:

1. a **surface-normal head** producing a 3-channel map. This output is L2-normalised along the channel dimension so that every pixel emits a unit vector $\hat{\mathbf{n}} = [n_x, n_y, n_z]^\top$.
2. a **segmentation head** producing 1-channel logits for the transparent-versus-background decision.

Both outputs are bilinearly upsampled to the input resolution during the forward pass. Training and evaluation use 256×256 inputs. During inference, the fully convolutional design permits arbitrary image resolutions.

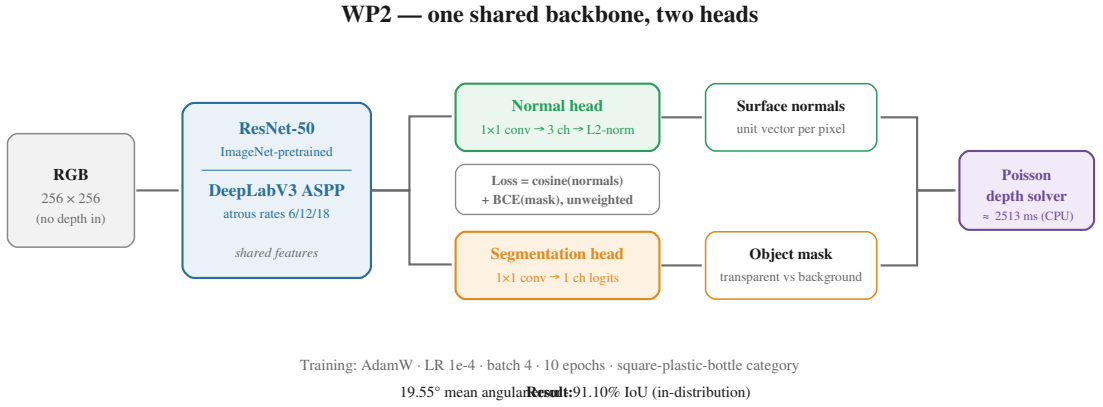


Figure 3.5: Perception module architecture.

3.2.2 Loss and Optimisation

The normal head is trained using a cosine similarity loss over all pixels,

$$\mathcal{L}_{\text{normals}} = 1 - \frac{1}{N} \sum_{i=1}^N \frac{\hat{\mathbf{n}}_i \cdot \mathbf{n}_i}{\|\hat{\mathbf{n}}_i\|_2 \|\mathbf{n}_i\|_2}, \quad (3.6)$$

and the segmentation head is trained using binary cross-entropy on the logits, $\mathcal{L}_{\text{mask}}$. The joint objective is the unweighted sum $\mathcal{L} = \mathcal{L}_{\text{normals}} + \mathcal{L}_{\text{mask}}$. We did not perform a weighting search because both losses converged at comparable scales. The network was trained for 10 epochs with AdamW (learning rate 10^{-4} , batch size 4) on the **square-plastic-bottle** category, using the normals and masks provided by ClearGrasp as training targets.

3.2.3 Depth Reconstruction Solver

The predicted normals and mask feed into a global optimisation process that reconstructs the metric depth over the transparent regions, as illustrated in Figure 3.6. This follows the strategy introduced by ClearGrasp [2]. The reliable depth values on the opaque background act as boundary anchors, and the predicted normals constrain the surface gradients inside the missing region.

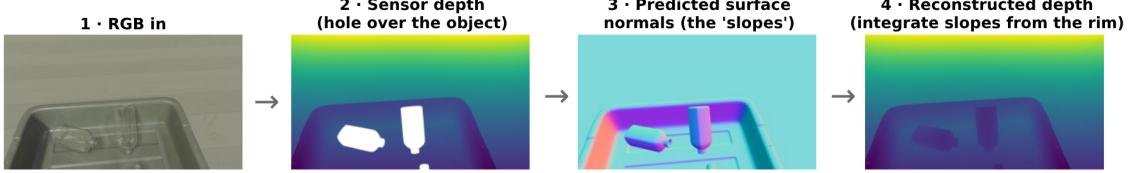


Figure 3.6: The depth reconstruction process under the ablation protocol (Section 3.4.2). From left to right: the RGB input; the simulated sensor failure where ground-truth depth is corrupted over transparent surfaces; the surface normals used to guide the solver; and the final reconstructed metric depth map.

Under the pinhole camera model, the depth gradient in pixel coordinates (u, v) relates to the surface normal as

$$\frac{\partial Z}{\partial u} = -\frac{n_x Z}{n_z f_x}, \quad \frac{\partial Z}{\partial v} = -\frac{n_y Z}{n_z f_y}, \quad (3.7)$$

where f_x and f_y are the focal lengths in pixels. We resolve the unknown depth Z by approximating it with the mean depth $\bar{Z}$ of the valid pixels in a two-pixel dilated band around the hole. This approximation works well because the object’s depth extent is small relative to its distance from the camera, which matches the geometry of a fixed overhead sensor.

The ClearGrasp renders use Intel RealSense D415 intrinsics ($f_x = f_y = 918$ pixels at the native 1920×1080 resolution). Focal lengths scale by the same factor as the image when downsampled (for example, $f_x = f_y = 459$ at the half resolution used for the depth ablation in Section 4.2). Omitting the $1/f$ scaling in Equation 3.7 inflates the gradient targets by more than two orders of magnitude and causes the solver to diverge.

Let Ω denote the hole region (the union of the predicted mask and any zero-depth pixels). The reconstruction $\mathbf{D}$ is the least-squares solution of a sparse linear system. This system applies finite-difference gradient constraints in u and v only to pixel pairs where at least one member lies in Ω , alongside the anchor constraints:

$$\lambda D(u, v) = \lambda D_{\text{raw}}(u, v), \quad (u, v) \notin \Omega, \quad (3.8)$$

using an anchor weight of $\lambda = 100$. Restricting the gradient constraints strictly to the hole region and weighting the anchors heavily prevents small mismatches between the predicted normals and true background geometry from altering the reliable depth measurements. Without these restrictions, the reconstruction degrades below the accuracy of the raw, corrupted input. The system is solved using the iterative LSQR method on a compressed

sparse-row matrix, and the output is clipped to the physically valid range of $[0, 2.5]$ m. Chapter 4 empirically measures how the choice of λ affects the final reconstruction.

OpenCV loads the ClearGrasp EXR normal maps in BGR channel order. This means the stored channels correspond to (n_z, n_y, n_x) rather than (n_x, n_y, n_z) . This channel re-ordering must be applied to both the ground-truth and predicted normals before solving Equation 3.7. Failing to do so silently exchanges the x and z components, producing divergent reconstructions.

3.3 Work Package 3: VLA Fine-Tuning

3.3.1 Base Model

The manipulation policy is OpenVLA-7B [4], an open-source Vision-Language-Action model that maps an RGB observation and a language instruction to discrete action tokens. Its vision stack fuses SigLIP and DINOv2 encoders, projecting their complementary features (global semantics and fine-grained spatial structure, respectively) into the embedding space of a Llama-2 7B language backbone. The actions are then generated autoregressively as text tokens. OpenVLA was selected over closed alternatives like RT-2 [5] because its weights, tokeniser, and action vocabulary are publicly available. These open components are strictly required for the fine-tuning and evaluation protocols used in this project.

3.3.2 Adapting Models Parameters with LoRA

Full fine-tuning of a 7-billion-parameter model is impossible on the available hardware (a single NVIDIA A100) and risks catastrophic forgetting of the pre-trained visuomotor skills. We use Low-Rank Adaptation [18] instead. The pre-trained weights $\mathbf{W}_0$ are frozen, and trainable low-rank updates are added to a selected set of projection matrices:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B}\mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times k}, \mathbf{B} \in \mathbb{R}^{d \times r}, \quad (3.9)$$

with rank $r = 32$ and dropout 0.05 held fixed throughout. We vary two parameters of the adaptation between runs (Section 3.3.4): the targeted projection matrices and the scaling parameter α . This tests whether earlier policy failures were caused by an under-powered training setup, not the training data itself.

To fit the optimiser state into GPU memory, the frozen base weights are stored in 4-bit NormalFloat (NF4) quantisation [19], while forward and backward computation is performed in `bfloat16`.¹

3.3.3 Training Protocol

Loss masking restricts supervision to the action tokens. For each sample, the label sequence is a copy of the input token sequence. Every position preceding the action tokens (located

¹We bypassed the `prepare_model_for_kbit_training` helper function because it caused a `bfloat16/float32` data type mismatch in a convolutional layer of the vision stack. Omitting it was necessary for training to proceed, and this approach was used for all three runs.

via `find_action_token_start`) and every padding position is set to the ignore index `-100`. This ensures the cross-entropy loss is computed over exactly the seven action tokens. Without this masking, the easily predictable prompt text dominates the loss, diluting the gradient signal on the actual coordinates that define the task.

Optimisation used AdamW with a batch size of 2, capped at five epochs. We computed the validation loss every 500 optimisation steps on a fixed subset of 25 validation samples (spanning approximately six scenes). Training included early stopping with a patience of 15 evaluations and automatically saved the best adapter checkpoint. Using a small, fixed validation subset was necessary to manage computational costs. Section 3.3.4 and Chapter 4 discuss the limitations of this choice for checkpoint selection.

Continuous validation monitoring is essential because an early experiment showed that without it, the model can memorise the training composites entirely undetected. In that test, the training loss reached machine zero, but spatial accuracy on held-out data remained at chance level. Chapter 4 analyses this failure mode and its diagnosis. The learning-rate schedule varied between runs and is detailed per run below.

3.3.4 Fine-Tuning Configurations: A Three-Run Sequence

The policy was fine-tuned in three successive configurations, each testing one potential cause of the failure detailed in Chapter 4. We discarded an initial attempt (Run 1) because a labelling defect caused the model to memorise constant coordinates (Chapter 4). We retain the run numbers to match the experiment logs. The three reported runs build upon each other regarding data, but maintain controlled training setups (Table 3.2). Run 2 establishes the baseline. Run 3 changes only the instruction distribution and fixes a prompt-formatting bug. Run 4 changes only the LoRA recipe, using Run 3’s data.

Run 2 (baseline). This run uses a single canonical instruction template (“Pick up the transparent plastic bottle.”). The LoRA update applies only to the query and value projections. This yields 16,777,216 trainable parameters (0.22% of the 7.56×10^9 total). The training uses a constant learning rate of 10^{-5} and an α scaling factor of 32 ($\alpha/r = 1$). A spacing bug in the prompt-assembly routine fused the final instruction word with the following token. This bug remained throughout Run 2 and was fixed in Runs 3 and 4. Training early-stopped at $\approx 39,500$ steps (≈ 2.6 epochs), hitting its best validation loss of 0.163 at step 32,000 after roughly 26 hours on one A100.

Run 3 (instruction diversification). This setup is identical to Run 2 except for two changes. First, we regenerated the training-split instructions from a pool of 139 phrasings (a 12-verb $\times$ 12-noun-phrase templating) while retaining the canonical phrase. The validation and test splits kept the single canonical string, and we reserved a disjoint set of seven held-out phrasings to evaluate how the model responds to instructions. Second, we corrected the prompt-spacing bug. The LoRA recipe (targets, learning rate, α) remained unchanged. Because the spacing fix applies only to the rewritten training split, Run 3 serves as a corrected retrain instead of functioning as a strict single-variable test. The

format-matched evaluation protocol (Section 3.4.4) controls for this spacing change.

Run 4 (fine-tuning recipe). This run uses the data from Run 3 but changes the LoRA recipe. This tests whether the attention-only, low-rate adaptation used in Runs 2 and 3 was too weak compared to the official OpenVLA recipe [25]. We implemented three changes. First, we applied the update to all seven linear projections per transformer block ($q, k, v, o, \text{gate}, \text{up}, \text{down}$) instead of just the query and value pair. Second, we raised α to 64 ($\alpha = 2r$, [19]). Third, we increased the learning rate to 2×10^{-4} with a 500-step warmup and a $10\times$ decay to 2×10^{-5} . Rank, dropout, quantisation, batch size, and the prompt-masked labelling were identical to Runs 2 and 3. This expands the trainable-parameter count from 16,777,216 (0.22%) to 79,953,920 (1.05%). Run 4 early-stopped at step 9,500, reaching the same validation-loss floor as Runs 2 and 3 (best 0.1732 at $\approx$ step 2,000) roughly fifteen times faster.

Checkpoint selection. For Runs 2 and 3, we used the adapter with the lowest validation loss. For Run 4, this same criterion selected a checkpoint that produced a normal validation loss but generated almost no parseable actions (Chapter 4). Therefore, we selected the reported Run 4 checkpoint manually based on its behavioural performance on held-out frames. Chapter 4 analyses how prefix-guided validation loss diverges from actual generation quality.

Table 3.2: Fine-tuning configurations. Rows above the rule are held constant across runs; rows below are varied deliberately to test the cause of the policy’s failure.

	Run 2 (baseline)	Run 3 (diversif.)	Run 4 (recipe)
Rank r	32	32	32
Dropout	0.05	0.05	0.05
Quantisation	NF4	NF4	NF4
Batch size	2	2	2
Label masking	prompt	prompt	prompt
Instruction set	canonical	139-pharse	139-pharse
Prompt spacing	fused	spaced	spaced
LoRA targets	q, v	q, v	all 7 linear
Scaling α	32	32	64
Learning rate	10^{-5}	10^{-5}	$2 \times 10^{-4} \dagger$

$\dagger$ 500-step warmup, then $10\times$ decay to 2×10^{-5} . “all 7 linear” = $q, k, v, o, \text{gate}, \text{up}, \text{down}$.

3.4 Evaluation Methodology

The evaluation answers four main questions. First, how accurately does the perception module recover geometry on both known and unseen object categories? Second, does using surface normals to rebuild depth data outperform basic image-filling methods, and how much do errors in those normals degrade the final depth? Third, how accurately does

the fine-tuned policy locate grasp targets, and do those predictions translate into physically plausible grasp points? Finally, what is the per-frame computational cost of the complete pipeline?

3.4.1 Perception Metrics

Surface-normal quality is measured using the mean, median, and root-mean-square angular error (in degrees) across object pixels. We also report standard threshold accuracies, which track the percentage of pixels with errors below 11.25° , 22.5° , and 30° . Segmentation performance is measured using the mean intersection-over-union. We compute all metrics in-distribution on the training category and out-of-distribution on two held-out ClearGrasp categories (`cup-with-waves` and `stemless-plastic-champagne-glass`). Because we did not have a held-out split of the training category, the in-distribution figures reflect how well the model fits the training data. The true test of shape generalisation relies on the model’s performance on the two unseen categories.

3.4.2 Depth Reconstruction Ablation

We simulate sensor failure by setting the ground-truth depth to zero over the object mask, ensuring the true depth of the corrupted region is known exactly. We compare four recovery conditions on 100 images at half resolution. Condition A uses the raw corrupted depth. Condition B uses Navier–Stokes inpainting (`cv2.inpaint`), representing a basic texture interpolation baseline with no 3D geometric awareness. Condition C applies the Poisson solver (Section 3.2.3) using ground-truth normals to establish the theoretical upper limit of the reconstruction. Condition D applies the same solver using the normals predicted by WP2, which represents the actual deployed configuration. We calculate all errors (RMSE, MAE, and mean relative error, in metres) strictly over the corrupted region.

Because these conditions are evaluated on the exact same images, we test for differences using the paired Wilcoxon signed-rank test, which avoids assuming the errors are normally distributed. Each condition’s mean RMSE includes a bootstrap 95% confidence interval using 10^4 resamples. We also sweep the anchor weight across $\lambda \in \{10, 50, 100, 500, 1000\}$ on a 30-image subset to measure how sensitive the reconstruction is to this specific hyper-parameter.

3.4.3 Policy Metrics

Policy accuracy is evaluated on the held-out validation split through autoregressive generation. The model receives the image and the instruction prompt, then outputs action tokens that are decoded back into coordinate bins. The primary metric is the mean L1 error (in bins) between the predicted and ground-truth coordinates, reported both per axis and in aggregate. We also report exact-match token accuracy. However, aggregate token accuracy can be highly misleading. Because five of the seven action dimensions are constants in this specific task, a policy can score a high overall accuracy even if it fails completely on the two spatial coordinates that actually matter. Therefore, per-axis coordinate error serves

as the definitive measure. We track accuracy across all training checkpoints to map out the learning trajectory.

To translate this bin-space error into physical task feasibility, we measure grasp success using three geometric criteria of decreasing strictness. The primary metric, ON-TARGET, counts a prediction as successful only if the point falls inside the segmentation mask of the specifically chosen target object. The ON-OBJECT metric relaxes this requirement to the mask of *any* object in the scene, which helps separate targeting errors from blind guesses on the empty background. The PROXIMITY metric counts any prediction falling within a fixed pixel radius of the ground-truth point, ignoring mask boundaries.

ON-TARGET acts as the headline measure throughout the evaluation because it is the only metric guaranteeing the predicted point lands on graspable material of the correct object. PROXIMITY serves purely as a diagnostic tool. Since a point can fall near the ground truth but still land on the bare conveyor belt, this metric provides an optimistic upper bound, not a realistic success rate. We report the on-target and on-object rates using confidence intervals resampled across scenes. Where reported, the looser proximity rate uses a standard Wilson interval.

The grasp-success proxy includes confidence intervals resampled across scenes. Because multiple composite samples share the same original source frame, we treat the scenes (rather than individual predictions) as the resampling unit. We perform 10^4 resamples over the 78 evaluation scenes ($n = 251$). This approach reflects the true effective sample size and produces a wider, more realistic interval than a naive per-sample calculation. Two baselines that ignore the image entirely put the coordinate error in context: a *centre-prior* that always guesses the exact centre of the image, and a *random* baseline that guesses uniformly across the frame. To prove it possesses genuine spatial grounding, the policy must beat the centre-prior, which is the strongest possible blind strategy on centred data. The margin of improvement over this prior measures the model’s actual spatial understanding, and the absolute size of that margin determines whether the prediction is precise enough for a physical grasp.

3.4.4 Instruction-Sensitivity Protocol

We evaluate the policy using a graded series of instructions to separate its ability to format text from its actual spatial understanding. A strict parser scores the results. An output counts as valid only if it matches the expected action-token format exactly. Any deviation triggers a parse failure and scores as a miss rather than being discarded. We report both the parse-failure rate and the on-target rate for each condition.

The test conditions range from the standard training phrase to held-out paraphrases with increasing semantic distance. For Run 4, we also include an absent-object instruction and a null instruction. This creates a total of eleven conditions, each evaluated over 200 samples. The evaluation tightly matches the formatting of the training data. Each run receives prompts using the exact spacing it saw during training. This ensures that any change in measured robustness comes from the model itself, not a mismatch in prompt formatting. We use a Bonferroni correction to assess statistical significance across conditions.

3.4.5 Base-Model Control

We run the un-fine-tuned OpenVLA-7B model as a positive control on real robot frames. This step proves whether the fine-tuned policy’s inability to follow different instructions comes from the fine-tuning process itself, and not from a flaw in the base model or the evaluation script. If the base model outputs different actions for different instructions on the exact same frame, the script is working correctly. Consequently, any loss of this ability after fine-tuning is caused directly by the adaptation. We score this control by counting the number of different actions the model produces across different instructions for a single frame, evaluated over 300 settings. We then compare this baseline against the fine-tuned policy.

3.4.6 Real-World Transfer: ZeroWaste and Perception Hit-Precision

We evaluate the synthetic-trained system on 572 real industrial frames from the ZeroWaste dataset [1]. Because ZeroWaste uses its own classification system, we map the transparent target to its `rigid_plastic` class. This mapping is imperfect because the class includes opaque items that the network never saw during training. We filter the dataset to evaluate only frames that contain this target class. The policy is scored using the same on-target and invalid-output rates as the synthetic evaluation.

Lacking per-pixel ground truth, the perception front-end is scored using *hit-precision*. This metric tracks whether the predicted mask activates on a frame containing the target, accompanied by the mask intersection-over-union against the mapped regions where they exist. Hit-precision measures whether the network degrades gracefully in the real world, instead of being penalised by mismatched labels that were not going to line up pixel-for-pixel.

3.4.7 Runtime Profiling

End-to-end latency is profiled on the A100 by timing each stage (perception forward pass, sparse depth reconstruction, and VLA generation) over 20 runs after warm-up, using CUDA synchronisation around the GPU stages. These results identify which component limits the pipeline’s throughput and quantify how close the system is to operating at real-world conveyor speeds.

3.5 Implementation and Hardware

All components were implemented in Python using PyTorch, HuggingFace Transformers, PEFT, OpenCV, and SciPy. Dataset generation runs on the CPU. Perception training, VLA fine-tuning, and all GPU evaluations ran on a single NVIDIA A100 on the university compute cluster, accessed through a jump host with sessions persisted in `tmux`.

The baseline VLA fine-tuning (Run 2) required approximately 39,500 optimisation steps, taking roughly 26 hours on the A100. The two subsequent runs are detailed in

Section 3.3.4. The depth reconstruction solver processes a 960×540 frame in approximately 2.5 s on the CPU.

3.6 Summary

This chapter detailed a three-stage pipeline driven by computer-generated training data. The synthetic data (WP1) trains both a geometric perception module (WP2) and a language-conditioned manipulation policy (WP3). A physics-consistent Poisson solver connects these systems by converting predicted surface normals into metric depth.

The most impactful design choices include per-sample action targets, background-identity data splitting, compositing-seam removal, prompt-masked loss computation, perspective-scaled gradient constraints, and continuous validation monitoring. Each of these decisions was motivated by concrete failure modes encountered and diagnosed during early development. The following chapter presents the quantitative results produced by the evaluation protocol defined in Section 3.4.

Chapter 4

Results and Analysis

This chapter evaluates the integrated system and its individual components following the protocol established in Section 3.4. Section 4.1 evaluates perception accuracy on both known and unseen objects. Section 4.2 details the depth-reconstruction ablation and anchor-weight sweep. Section 4.3 analyses the VLA policy, diagnosing how the model memorised data in the initial configuration, tracking the corrected training dynamics, and comparing spatial accuracy against a positional baseline. Section 4.4 connects this spatial accuracy to physical grasp feasibility and identifies data quality as the primary limitation. Section 4.5 tracks instruction robustness across the three fine-tuning runs. Section 4.6 uses a base-model control to prove the fine-tuning process caused the policy to ignore instructions. Section 4.7 evaluates real-world transfer on ZeroWaste industrial frames. Finally, Section 4.7.1 details a finding regarding checkpoint selection, and Section 4.9 profiles the system’s runtime.

4.1 Perception Module

Table 4.1 summarises the perception module’s surface-normal and segmentation accuracy. The in-distribution rows computed over the full **square-plastic-bottle** category (11,701 images) and overlaps with the network’s training data. This should be treated as an upper bound, not a generalisation estimate. The two out-of-distribution (OOD) rows evaluate categories the network never observed, providing a direct measure of its ability to generalise.

Table 4.1: Perception accuracy in and out of distribution. Angular errors in degrees over object pixels; threshold columns give the fraction of pixels with error below the stated angle. ID = in-distribution (overlaps training data); OOD = held-out category.

Category	Split	N	Mean	Median	RMSE	<11.25°	<22.5°	IoU
square-plastic-bottle	ID	11,701	19.55°	14.29°	25.57°	39.3%	69.4%	91.10%
cup-with-waves	OOD	100	34.06°	22.99°	44.03°	21.0%	49.1%	77.28%
champagne-glass	OOD	117	25.53°	18.00°	33.71°	27.8%	60.8%	77.87%

The error distribution in Figure 4.1 is unimodal and right-skewed. The network estimates most object pixels well (median 14.3°). The 19.55° mean is pulled upward by

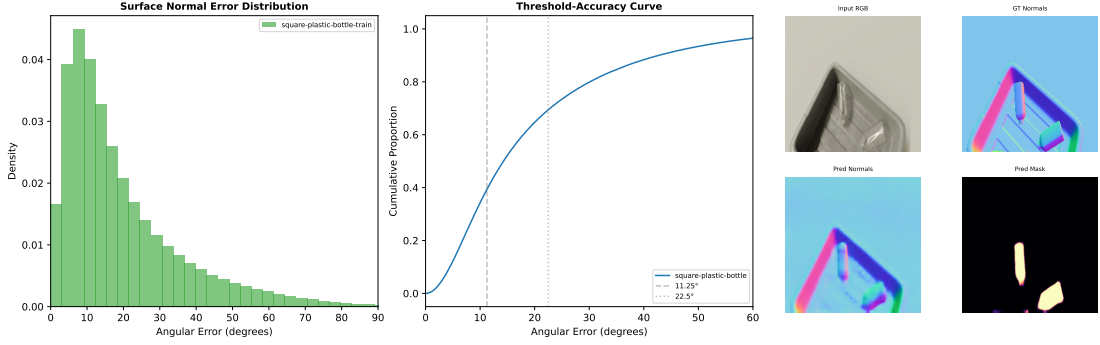


Figure 4.1: In-distribution perception behaviour on the training category. Left: surface-normal angular-error distribution, right-skewed with its mode near 8° and a long tail beyond 40° . Centre: cumulative threshold-accuracy curve with the standard 11.25° and 22.5° thresholds marked. Right: a qualitative example (input RGB, ground-truth normals, predicted normals, predicted mask) showing accurate object localisation with smoothed normal detail. Out-of-distribution results are tabulated in Table 4.1.

a minority of high-error pixels concentrated at object boundaries and internal refractive details, as shown in the qualitative panel. In these areas, the prediction is visibly smoother than the ground truth.

Two observations appear from Table 4.1. First, segmentation generalises better than geometry. The IoU falls by roughly 14 percentage points on unseen categories (from 91.1% to $\sim 77.5\%$). While the mean angular error grows by 31% to 74% depending on the target shape. The network still locates unfamiliar transparent objects reliably, but it estimates their surface orientation with less accuracy.

Second, this geometric degradation depends heavily on shape. The stemless champagne glass is tall, smooth, and convex. Because it is geometrically similar to the training bottles, it loses only 6° of mean accuracy. In contrast, the wavy-edged cup has a corrugated surface unlike anything in the training data, causing it to lose 14.5° of accuracy. Generalisation relies more on geometric similarity to the training data than on whether the object is transparent.

4.2 Depth Reconstruction Ablation

Table 4.2 presents the four-way ablation over 100 images, with errors computed exclusively over the corrupted (transparent) region. Figure 4.2 shows representative reconstructions together with the per-method error distributions; Figure 4.3 isolates the mean RMSE with bootstrap confidence intervals.

Paired Wilcoxon signed-rank tests over the matched images yield the following results. Both recovery methods improve on the raw input ($p < 10^{-17}$). Between the solver driven by *ground-truth* normals and the inpainting baseline, the difference is not significant (median difference -0.1 mm, $p = 0.64$). With the correct camera focal length, the Poisson solver given perfect normals is therefore *statistically indistinguishable* from a strong baseline that ignores 3D geometry. It falls marginally behind on RMSE but ahead on MAE (0.1091 vs 0.1102 m) and relative error (0.1378 vs 0.1404). The solver is thus correctly

Table 4.2: Depth reconstruction error over the corrupted region (100 images, half resolution, corrected camera focal length $f_x = f_y = 693$ at 960×540). RMSE with bootstrap 95% confidence intervals from 10^4 resamples; MAE and mean relative error given for completeness.

Method	RMSE (m)	95% CI	MAE / Rel
(A) Raw corrupted depth	0.9297	[0.8646, 0.9965]	0.9274 / 1.000
(B) Navier–Stokes inpainting	0.1227	[0.1152, 0.1306]	0.1102 / 0.140
(C) Poisson + GT normals	0.1288	[0.1193, 0.1390]	0.1091 / 0.138
(D) Poisson + pred. normals	0.3379	[0.3188, 0.3575]	0.2292 / 0.276

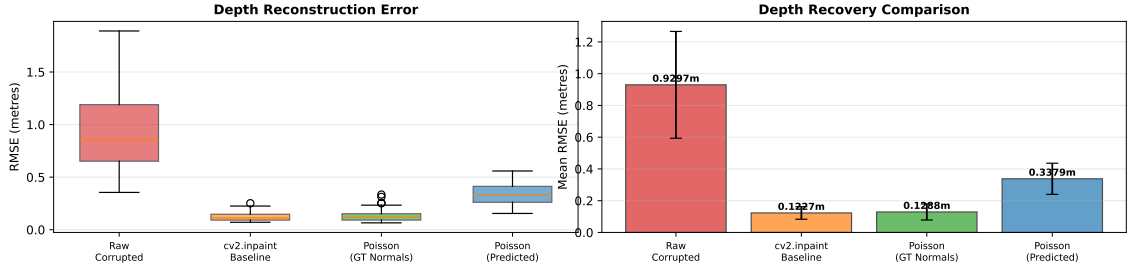
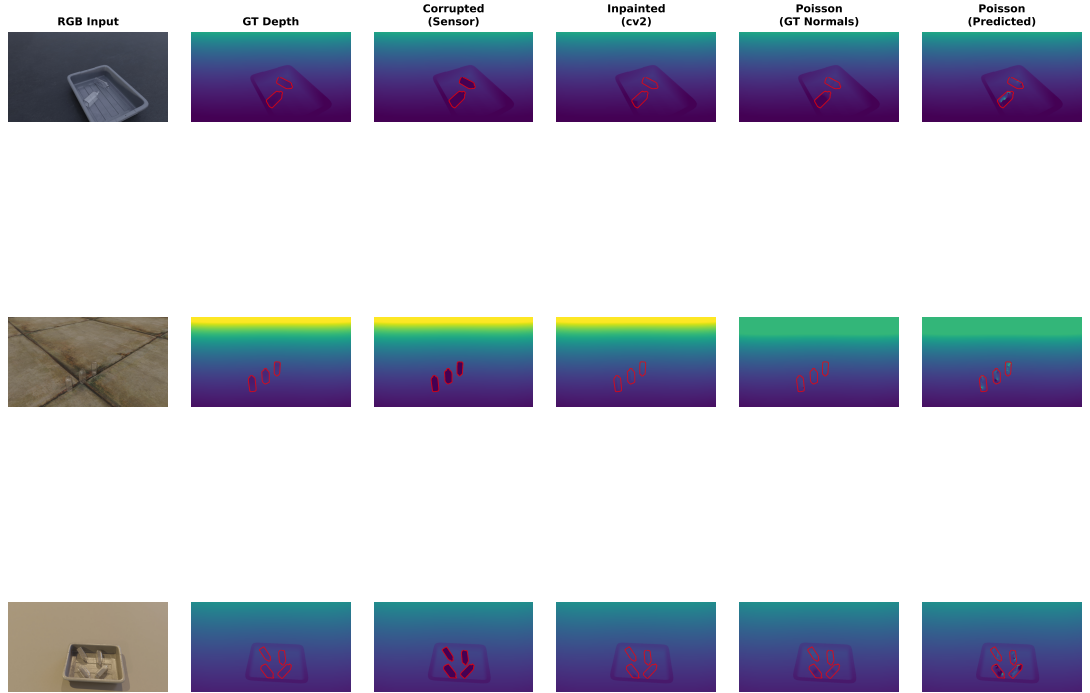


Figure 4.2: Depth reconstruction ablation. Top rows: qualitative comparisons on three scenes (RGB input, ground-truth depth, corrupted sensor depth, inpainting baseline, Poisson with ground-truth normals, Poisson with predicted normals), with red contours marking the transparent regions; residual artefacts inside the contours are visible only in the predicted-normals condition. Bottom: RMSE box plots per method (note the outliers in the ground-truth-normals condition) and mean-RMSE bar chart.

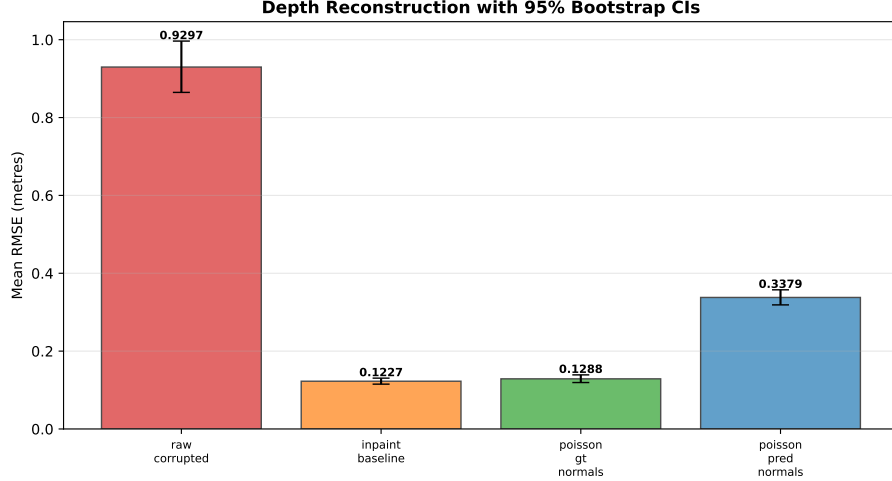


Figure 4.3: Mean RMSE per reconstruction method with bootstrap 95% confidence intervals.

implemented: given ideal input it matches inpainting to within measurement noise. The deployed configuration’s deficit stems entirely from perception error. Driving the same solver with *predicted* rather than ground-truth normals raises RMSE by 0.21 m (median difference 209.6 mm, $p < 10^{-4}$), providing a clean quantification of how perception errors degrade metric depth.

The statistical parity of conditions B and C confirms the solver is correctly implemented: with perfect input it matches inpainting, so condition D’s deficit is caused by perception error, not solver error. Consequently, normal-guided reconstruction is *not justified by these results for the shallow object shapes evaluated*. It equals inpainting under ideal normals and falls well behind under real ones, while consuming the majority of pipeline latency (Section 4.9). Normal integration is expected to earn its cost only where object depth varies substantially relative to the surroundings (such as deep containers or stacked items), which remain untested here. Within this evaluation, the choice for shallow objects remains the inpainting path due to equal accuracy at lower cost.

4.2.1 Anchor-Weight Sensitivity

Table 4.3 and Figure 4.4 report the sensitivity of condition C to the anchor weight λ over a 30-image subset. Reconstruction error is flat across $\lambda \in [10, 100]$ (0.1247 to 0.1229 m, a 1.5% spread) and degrades sharply beyond it (79% at $\lambda = 500$, 249% at $\lambda = 1000$). At these higher values, over-weighted anchors suppress the in-hole gradient constraints. The method is thus insensitive to λ within this operating order of magnitude. The value $\lambda = 100$ sits at the favourable end of this stable plateau and was not tuned to the test data.

Table 4.3: Reconstruction RMSE (condition C) as a function of anchor weight λ (30 images).

λ	10	50	100	500	1000
RMSE (m)	0.1247	0.1239	0.1229	0.2195	0.4290
Std (m)	0.0461	0.0449	0.0402	0.0822	0.1168

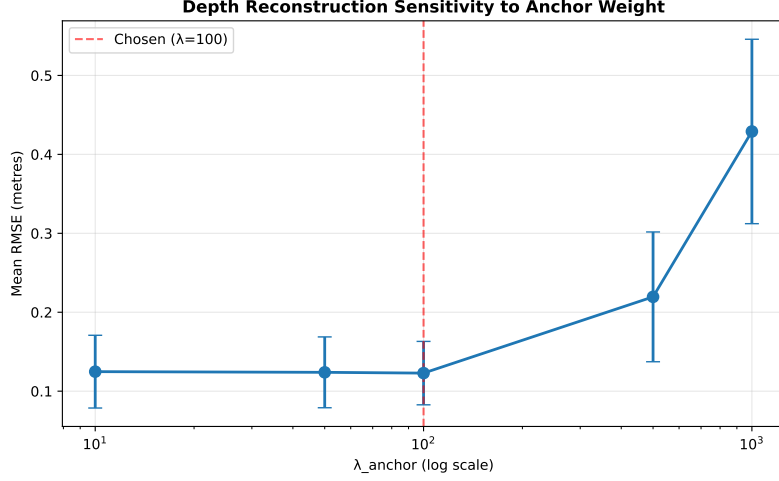


Figure 4.4: Reconstruction RMSE against anchor weight λ (log scale, error bars one standard deviation). The chosen operating point $\lambda = 100$ lies at the end of a flat plateau spanning $\lambda \in [10, 100]$.

4.3 Policy Learning

4.3.1 Diagnosing How the Model Memorised Data

The initial fine-tuning configuration contained a data-pipeline defect. Due to a fault in the dataset loader, every training sample was paired with a single fixed action vector instead of its correct per-image grasp target. Despite this fundamental error, the metrics looked promising. Training loss fell from 26.08 to exactly zero by step 4,480, and aggregate action-token accuracy reached 97.71%. However, inspecting individual generations revealed the model emitted the exact same coordinate pair for every input image. It had simply memorised the constant label and learned nothing about the images. (An early evaluation script recorded a spatial error of 47 bins for this model, but this resulted from a token-slice misalignment bug and is excluded from the quantitative analysis below).

Aggregate token accuracy is a misleading metric for VLA action strings where most dimensions are constants. Because five of the seven action tokens in this task are fixed parameters, a policy that fails completely on the two spatial coordinates can still score above 70% just by learning the formatting, and near-perfectly if it memorises the constants. Therefore, per-coordinate error must be reported separately. Furthermore, a training loss reaching exactly zero should be treated as a red flag. It proves that the training set is easily memorisable. For a spatial task, this implies the data lacks the diversity required to force actual perceptual learning. These findings motivated the loss masking, per-sample target verification, and continuous validation monitoring implemented in Section 3.3.3.

4.3.2 Corrected Training Dynamics

Once the per-image targets, background-identity splitting, and the full augmentation suite (Section 3.1.3) were in place, the training dynamics changed qualitatively. Loss fell from 25.4 and plateaued near 0.15–0.17 without ever approaching zero. Validation loss tracked the training loss closely throughout, which means the residual error is task difficulty rather than overfitting.

Early stopping halted training at approximately step 39,500 (≈ 2.6 epochs, ≈ 26 h on one A100) with a best validation loss of 0.163 at step 32,000. The fact that the model plateaus at this floor instead of continuing to fall, indicates a ceiling set by the data. The residual error is bounded by the fidelity of the training data, not by the training budget. Run 4 corroborates this conclusion by reaching the exact same floor roughly fifteen times faster (Section 4.7.1).

4.3.3 Spatial Accuracy

Policy accuracy was evaluated by autoregressive generation on held-out validation composites across all saved checkpoints (50 samples per checkpoint). We excluded checkpoints that produced more than five unparseable outputs from the trend analysis, as parser contamination artificially inflates their per-axis means. Table 4.4 compares the spatial accuracy of the two training iterations, and Figure 4.6 shows the learning trajectory of the second run.

Table 4.4: VLA spatial accuracy across the two training iterations. Clean checkpoints only (parse failures ≤ 5).

	Run 1	Run 2
Training data	composites only (640×480)	50/50 composite + native (224×224)
Label validity	90.0%	100%
Dataset size	24,570 train	30,850 train
Best-checkpoint L1 (bins)	27.60	20.59
Best-checkpoint median L1	26.50	21.00
X error (bins)	32.70	23.98
Y error (bins)	32.46	27.47

The best clean checkpoint by spatial L1 error (step 20,500) achieves a mean spatial L1 error of **20.59 bins** (median 21.0), with per-axis coordinate errors of 23.98 (x) and 27.47 (y). This checkpoint is different from the best-validation-loss checkpoint at step 32,000, because validation loss and generation accuracy do not always align, an issue that is explored further in Section 4.7.1 for Run 4. The 20.59-bin result represents a 25% reduction in mean L1 error compared to the best performance of the first run (27.60 bins). Multiple nearby checkpoints perform comparably (steps 16,000, 22,000, 22,500, and 30,500 all fall in the 21–22 bin range). This indicates a stable convergence plateau and not just a single lucky checkpoint.

The policy’s spatial competence is bimodal, meaning a mean summary alone is misleading. On the matched validation set ($n = 251$), the model’s median distance to the

ground-truth grasp point is 42.2 px, which beats the centre-prior baseline of 47.2 px (visualised in Figure 4.5). On a typical sample, the policy localises the object better than simply guessing the image centre, demonstrating genuine (if imprecise) spatial grounding.

However, the mean distance inverts this ordering: the model averages 64.0 px against the centre-prior’s 50.7 px. This happens because 26% of the predictions (65/251) miss by more than 100 px. These gross mislocalisations, which sometimes stretch across the entire frame diagonal, artificially inflate the mean without reflecting the model’s typical behaviour. When restricting the analysis to the 186 non-tail predictions (≤ 100 px), the model’s mean error falls to 35.8 px, sitting well inside the centre-prior. The policy therefore does not fail uniformly. It successfully grounds the target on roughly three-quarters of the samples and mostly fails on the remainder. This bimodality recurs throughout the evaluation, appearing in both the instruction-shift response (Section 4.5) and the qualitative grasp galleries. This pattern is typical of a model that has learned a targeting zone but lacks robust precision.

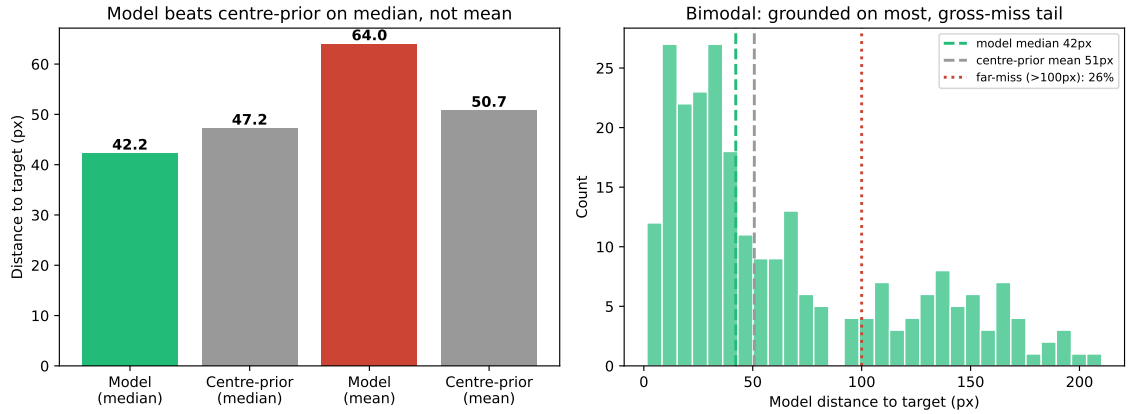


Figure 4.5: Policy spatial accuracy against a centre-prior baseline, matched sample-for-sample ($n = 251$). **Left:** Mean and median pixel distance to the ground-truth target. **Right:** Histogram of the model’s spatial error distribution, highlighting the subset of predictions exceeding a 100-pixel threshold.

The per-axis asymmetry ($X \approx 24$, $Y \approx 27\text{--}45$ across checkpoints) highlights a structural bias. The Y error is consistently higher and more variable than X. This likely reflects the vertical composition bias of the ClearGrasp renders, where objects typically sit in the lower portion of the frame, combined with the reduced vertical extent of the objects at the 224×224 resolution.

4.4 Grasp-Feasibility Measurement

4.4.1 Label Audit

A mask-based evaluation of the first training iteration revealed a defect in the training labels. Only 90.0% of the ground-truth targets actually fell inside their target mask (171/190). The cause is geometric. For concave or crescent-shaped objects, the contour centroid used to calculate the target often falls outside the object boundary. Consequently,

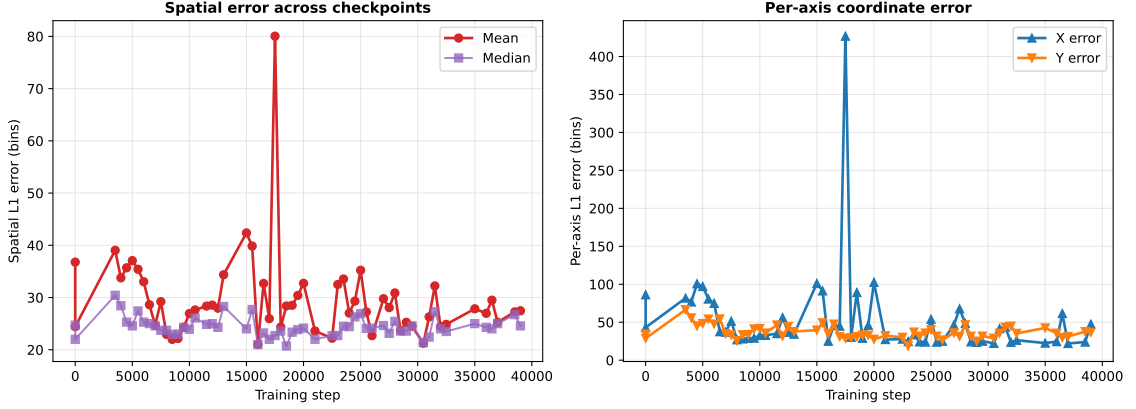


Figure 4.6: Run 2 spatial L1 error (left: mean and median) and per-axis coordinate error (right) across training checkpoints. The mean is volatile due to parser contamination at some checkpoints; the median (purple) is stable and falls to ~ 21 bins by step 8,000, remaining there through 39,000 steps.

roughly one in ten training samples across the 24,570 composites taught the policy to target the empty background. This label noise contaminated the training process and artificially lowered the measured success rates. We corrected this using point-in-polygon validation with a distance-transform fallback (Section 3.1.2). This fix represents one of the three data revisions in the second training iteration, guaranteeing 100% label validity.

4.4.2 Run 1 Baseline

Under exact mask evaluation, the first-generation model (190 parseable validation samples) achieves a 2.6% ON-TARGET rate (Wilson 95% CI [1.1%, 6.0%]), a 4.7% ON-OBJECT rate ([2.5%, 8.8%]), and a 14.7% PROXIMITY rate (40 px radius, [10.4%, 20.5%]). Comparing these metrics reveals that the proximity figure is actually an optimistic upper bound rather than a conservative lower bound. Most predictions within the 40-pixel disc land on background pixels adjacent to the object rather than on the plastic itself. This results in the first-generation policy failing to target solid object material in over 97% of cases.

4.4.3 Run 2: Effect of Data Revisions

We applied three fundamental data revisions and retrained the policy from scratch. First, we implemented the target derivation method described in Section 3.1.2, which ensures all coordinates are valid, to eliminate label noise. Second, we used a 50/50 mixture of composites and native ClearGrasp frames. Third, we generated the data directly at the encoder-native 224×224 resolution. Table 4.5 compares the two runs under identical mask-exact evaluation.

The ON-TARGET rate nearly tripled (from 2.6% to 7.2%), and the ON-OBJECT rate similarly increased (from 4.7% to 12.7%). These metrics remain below deployment-grade precision. However, the improvement is statistically significant, as Run 2 ON-OBJECT confidence interval [9.2%, 17.4%] does not overlap with the Run 1 interval [2.5%, 8.8%].

Table 4.5: Grasp-feasibility comparison between training iterations under identical mask-exact evaluation. Wilson 95% confidence intervals in brackets. ON-TARGET rates use the scene-clustered bootstrap interval; ON-OBJECT and per-cell breakdowns retain Wilson intervals.

	Run 1 ($n = 190$)	Run 2 ($n = 251$)
Label validity	90.0%	100%
Training mix	composites only	47% composite / 53% native
Resolution	640×480 → 224	native 224×224
Mean spatial L1 (bins)	27.6	20.6
ON-TARGET	2.6% [1.1, 6.0]	7.2% [3.9, 10.9]
ON-OBJECT	4.7% [2.5, 8.8]	12.7% [9.2, 17.4]

4.4.4 Breakdown by Sample Type

Table 4.6 breaks down the performance split between native ClearGrasp frames and composites within the Run 2 validation set.

Table 4.6: Run 2 grasp accuracy by sample type. Wilson 95% CIs in brackets.

Sample type	n	ON-TARGET	ON-OBJECT
Native ClearGrasp	124	11.3% [6.8, 18.1]	20.2% [14.0, 28.1]
Composite	127	3.1% [1.2, 7.8]	5.5% [2.7, 10.9]

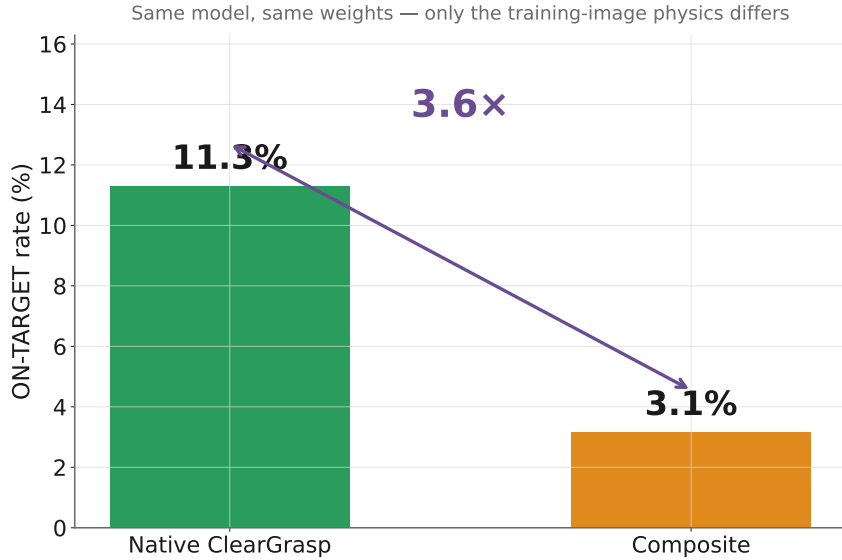


Figure 4.7: Grasp on-target rate by sample type (native ClearGrasp frames versus composited frames) evaluated on the Run 2 policy.

The model is nearly four times more accurate on native frames (11.3% on-target) than on composites (3.1%) as highlighted in Figure 4.7. This confirms the hypothesis from Section 3.1.4: compositing removes the refraction, shadows, and specular highlights that carry spatial information for transparent objects. Restoring those physical cues via native renders measurably improves localisation. Furthermore, the composite performance in

Run 2 (3.1%) matches the full-dataset performance of Run 1 (2.6%). This indicates that the Run 2 improvement stems almost entirely from adding native frames to the training mix, not from the label or resolution fixes alone.

When bootstrapping across scenes (Section 3.4.3), the native–composite difference is +8.1 percentage points, with a 95% CI of [1.1, 15.9]. Because this interval excludes zero, the native-frame advantage is statistically significant. It is not just a numerical gap between two overlapping per-cell Wilson intervals. This trend remains consistent across all three training runs (Section 4.4.6). It shows that how realistic the compositing is, not model capacity, sets the spatial ceiling..

4.4.5 Breakdown by Object Size

Table 4.7 and Figure 4.8 report grasp accuracy across object-area terciles. The model cannot localise small objects at all (0% on-target), achieves marginal accuracy on medium objects (7.3%), and performs best on large objects (14.1% on-target, 20.0% on-object). At the 224×224 working resolution, “small” objects occupy fewer than 142 pixels of area. This footprint is too small to resolve at the VLA encoder’s patch granularity.

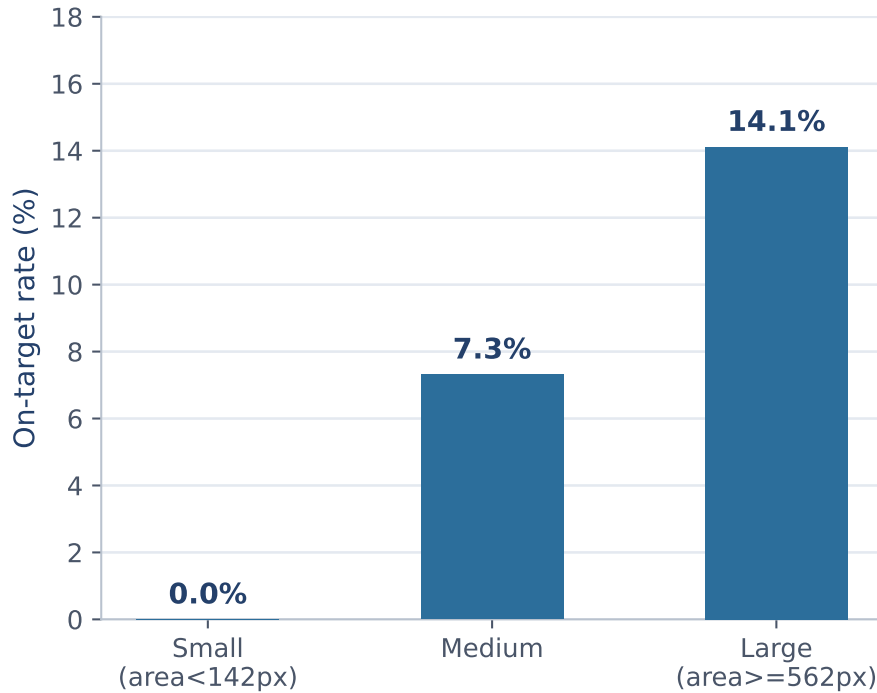


Figure 4.8: Run 2 grasp-target predictions by object-area tercile.

4.4.6 The Spatial Ceiling Holds Across Runs

Neither instruction diversification nor the recipe fix raised spatial accuracy. Run 3 achieves 6.8% on-target overall, with confidence intervals overlapping Run 2’s 7.2%, and preserves the native/composite gap (9.8% vs 3.9%). Run 4 at step 9000 achieves 2.7% on-target overall (8/300), with 4.5% [2.2, 9.0] on native frames and 0.7% [0.1, 3.8] on composites.

The overall Run 4 figure falls just below the 4–8% band pre-registered for this checkpoint. This drop is concentrated in the composite subset (0.7%), a condition that remains at the performance floor in every run. On native frames (which carry diagnostic weight because they retain transparency cues), Run 4’s 4.5% overlaps the confidence intervals of Run 2 (11.3%) and Run 3 (9.8%). Therefore, the data does not support a claim of regression on valid images.

The three runs establish that spatial accuracy remains at a statistically indistinguishable, low level across all configurations. No run achieves an overall rate above single digits, and none get past the ceiling set by the data. The improved training recipe repaired the *format* of the output (Section 4.5) but left *spatial* accuracy unchanged. This confirms that format generation and spatial grounding are independent. The formatting was fixable through hyperparameter tuning, but spatial accuracy requires better training data.

Table 4.7: Run 2 grasp accuracy by object-area tercile. Wilson 95% CIs in brackets.

Size band	n	ON-TARGET	ON-OBJECT
Small (< 142 px)	84	0.0% [0.0, 4.4]	2.4% [0.7, 8.3]
Medium	82	7.3% [3.4, 15.1]	15.9% [9.5, 25.3]
Large (≥ 562 px)	85	14.1% [8.3, 23.1]	20.0% [12.9, 29.7]

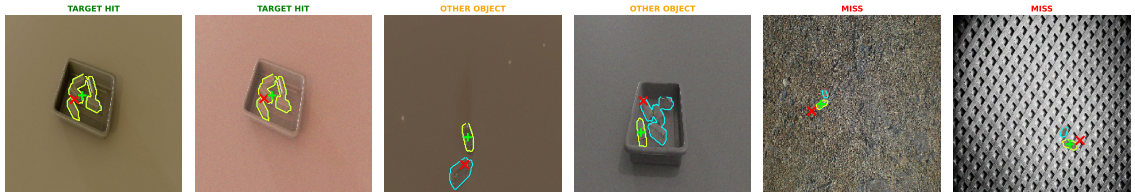


Figure 4.9: Run 2 grasp-target predictions. Left two: TARGET HIT (prediction on the selected object). Middle two: OTHER OBJECT (prediction on a different item in the cluster). Right two: MISS (prediction on background). Yellow contours: target object; cyan contours: all objects; red cross: prediction; green cross: ground truth.

4.5 Instruction Robustness and Semantic Grounding

The grasp evaluation reveals *where* the policy places its target (with qualitative examples of these placements shown in Figure 4.9). This section examines whether that placement is *controlled by the instruction*. We evaluate the three fine-tuning runs using the graded, format-matched protocol from Section 3.4.4. The results are scored with the strict pre-registered parser, which treats any malformed output as a miss ($n = 200$ per condition).¹

Run 2: off-template collapse. On the single canonical training template, the policy’s outputs are well-formed. When the instruction is paraphrased away from that baseline, the parse-failure rate rises from 18% to 76%, with malformed outputs degrading into

¹A preliminary, more lenient parser produced instruction-sensitivity figures that did not reproduce under the strict parser. All results in this section use the strict parser exclusively.

multilingual token gibberish. On a paired test evaluating whether the policy follows a *category-specific* instruction (whether the predicted target moves toward the named object), discrimination sits exactly at chance (50%).² The instruction does not actually drive the behaviour. The policy simply emits a fixed spatial pattern and its formatting breaks when prompted off-template.

Run 3: the failure mode shifts, the rate does not. Training on the 139-phrase instruction pool (Section 3.3.4) leaves the overall parse-failure *rate* essentially unchanged (25% to 78% across the same gradient). However, it changes the *character* of the failures. Off-template outputs are no longer gibberish; they become near-valid, degraded coordinate strings (e.g., a truncated `131>121</s>`). A classification of these failures shows roughly 75% are near-valid at baseline, declining to 0–25% under the most severe instruction shifts. Diversifying the instructions taught the model the *surface form* of the action grammar without actually grounding the semantics.

Run 4: the format is repaired; grounding does not appear. The model trained with the corrected recipe, evaluated at the step-9000 checkpoint (Section 4.7.1), produces exactly *zero* parse failures across all eleven conditions (0/2,200 generations, 95% CI [0, 1.9%]). The output formatting is fully repaired. Nevertheless, the on-target rate remains flat at roughly 4–8% across *every* condition, including an absent-object instruction and a null instruction. The highest-performing cell is the generic “Pick up the object” at 8%. After applying a Bonferroni correction, no condition differs significantly from any other. The policy exhibits *instruction-blind targeting* by placing its target using the same spatial distribution regardless of the prompt. The flat accuracy on absent-object and null prompts proves directly that instruction content is ignored during computation.

Synthesis. Across the three runs (summarized visually in Figure 4.10), instruction diversification and a stronger LoRA recipe repaired the *format* of the policy’s output. The outputs progressed from off-template gibberish, to degraded-but-near-valid strings, to fully valid tokens under every prompt. However, semantic grounding remained absent. The policy learned to emit an action *pattern*, it did not actually understand the scene or the instruction. The instruction itself never drove the physical targeting behaviour. This finding motivated the base-model control in Section 4.6 to test whether this insensitivity stems from the base model itself or from the fine-tuning process.

4.6 Base-Model Control: Fine-Tuning Caused the Instruction Blindness

The instruction-blindness of Section 4.5 could in principle be a property of OpenVLA itself or of the evaluation harness, rather than a result of the fine-tuning. The positive control

²An earlier draft reported 61.8% here. That figure was an artifact of a prompt-formatting defect (a literal `\n` in the evaluation prompt assembly). With the defect corrected, discrimination is at chance. The corrected value is used throughout.

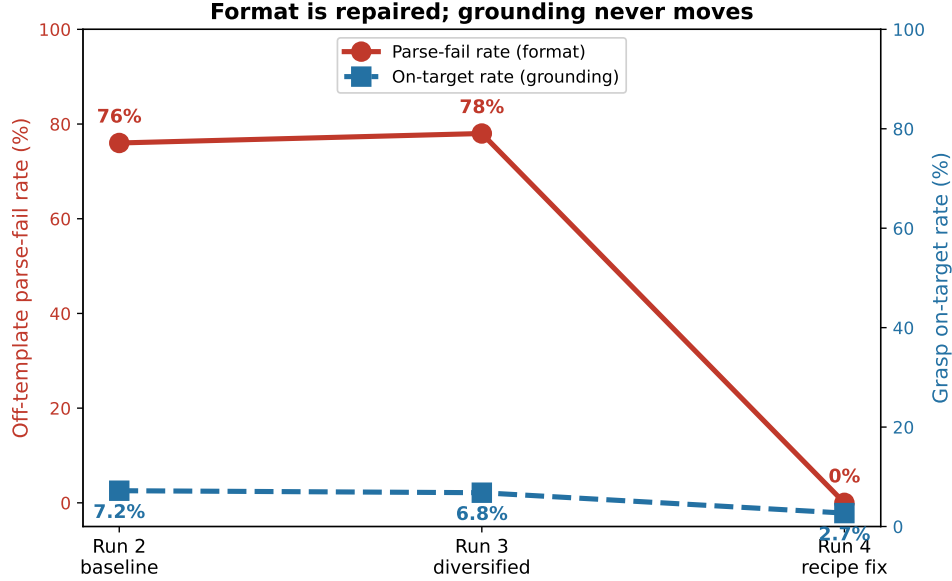


Figure 4.10: Comparison of off-template parse-failure rates (red) and on-target grasp rates (blue) across the three fine-tuning configurations. Parse failures measure output formatting validity, while on-target rates measure semantic spatial grounding.

of Section 3.4.5 rules this out. On real robot frames, the un-fine-tuned OpenVLA-7B is fully instruction-sensitive. It emits an average of 4.0 different actions across different instructions per frame ($n = 300$). This proves the evaluation harness correctly detects instruction sensitivity, and the base model naturally responds to varying instructions. By contrast, the fine-tuned policy emits only 1.80 different valid points on the same measure. The fine-tuning process therefore directly caused the loss in instruction sensitivity. This conclusion is measured against a passing positive control, and not simply inferred from the fine-tuned model’s behaviour on its own.

4.7 Real-World Transfer: ZeroWaste

The synthetic-trained system was evaluated on 572 real material-recovery-facility frames from ZeroWaste [1], following the protocol of Section 3.4.6. The two components fail in opposite ways. On one hand, the policy degrades completely. It achieves 0% on-target accuracy and produces almost exclusively malformed output (multilingual gibberish), placing it well below even the centre-prior baseline.

On the other hand, the perception front-end remains effective. It fires on 100% of the frames with a hit-precision of 30%, maintaining a mask IoU of 8.8% against the mapped `rigid_plastic` regions. This low overlap is expected because the ZeroWaste class includes opaque items the network was never trained on. However, the front-end continues to successfully *localise* transparent material on genuinely out-of-domain imagery, whereas the policy produces no usable output at all. As such, the geometric front-end carries a transferable signal into the real world, but the text-serialised policy does not.

4.7.1 Validation Loss Is a Misleading Selection Signal

Run 4 exposed a major flaw in the training-time monitoring strategy. The adapter with the lowest validation loss computed against the ground truth (0.1732, at $\approx$ step 2,000) failed to produce a parseable action in 94–99.5% of cases during autoregressive generation. A checkpoint can look converged under the early-stopping loss while failing entirely in deployment.

Consequently, the step-9000 checkpoint reported throughout this chapter was selected *post-hoc* by behavioural evaluation on held-out frames, not by validation loss. For action models that serialise their output as text, standard validation loss and actual generation quality can diverge sharply. Relying on loss alone for checkpoint selection is highly unsafe. This flaw severely compounds the limits of using a small, fixed validation subset (Section 3.3.3).

4.8 End-to-End Qualitative Results

Figure 4.11 traces the full pipeline on four native ClearGrasp scenes: RGB observation, predicted surface normals, reconstructed metric depth, and the VLA’s grasp target overlaid with the object boundaries. The normals accurately localise the transparent objects, and the reconstructed depth is physically plausible across both tote and floor scenes.

Furthermore, these images constitute a *domain shift for the policy*, which was trained on ReSort-IT composites rather than native renders. Its targets land near, but frequently not on, the intended objects (upper rows). This performance aligns with, and is slightly worse than, the in-domain quantitative results from Section 4.4.

4.9 Runtime Profiling

End-to-end latency was profiled on the A100 over 20 warm runs per stage, using CUDA synchronisation around the GPU stages. The total per-frame time is approximately 2.98 s (a throughput of 0.34 FPS). This breakdown (charted in Figure 4.12) clearly identifies which is actually slower. The CPU-bound sparse depth solver accounts for roughly 63% of the frame time, autoregressive VLA generation takes about 36%, and the perception forward pass requires under 1%. The 7B language model is not what is slowing throughput down, the depth solver is.

As shown in the ablation study (Section 4.2), simple inpainting matches the solver’s accuracy on these shallow objects. Therefore, replacing the CPU solver with an inpainting pass is the single highest-leverage runtime optimisation. It roughly halves the per-frame cost with no measured accuracy loss for the evaluated shapes. While the absolute rate remains far below conveyor-speed operation, this profiling effectively exposes the computational costs of each pipeline stage.

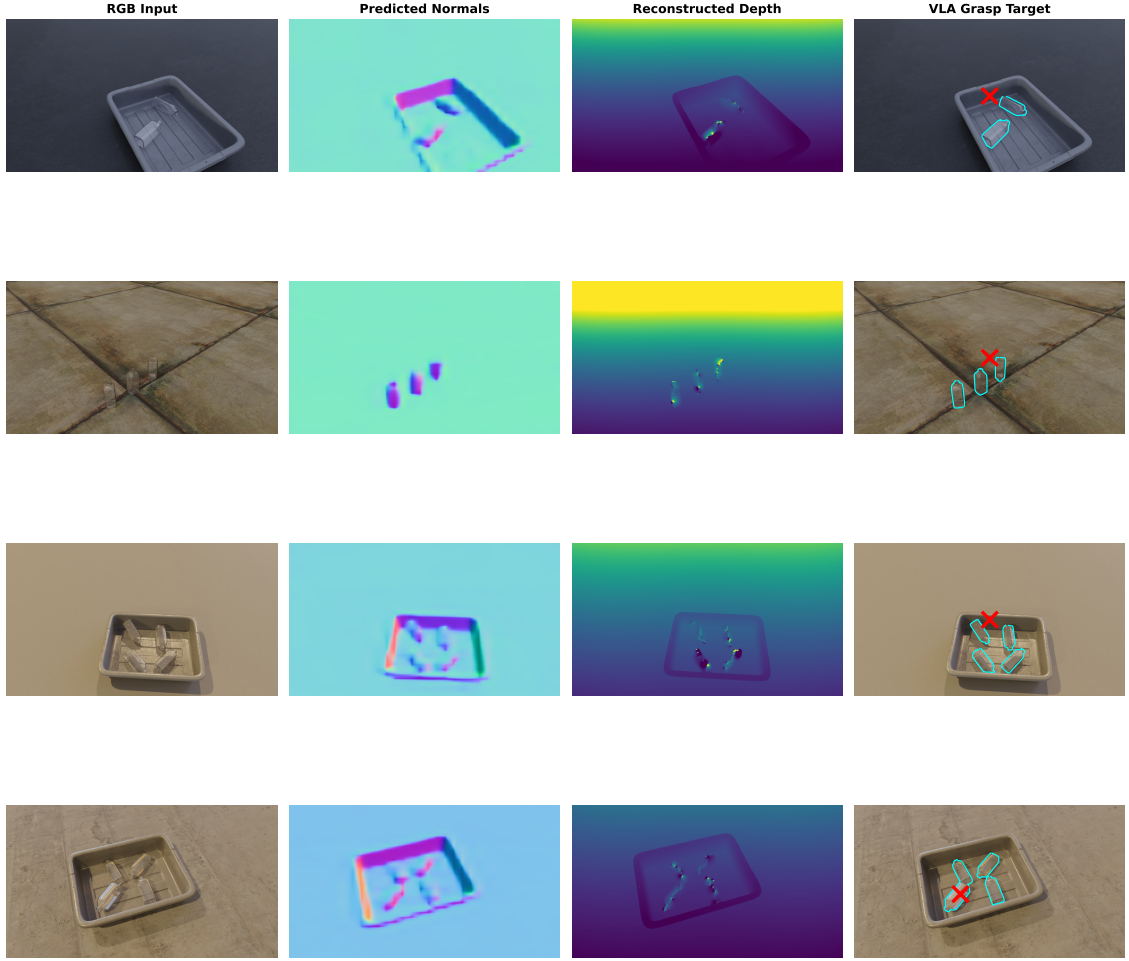


Figure 4.11: End-to-end pipeline outputs on four native ClearGrasp scenes (a domain shift from the policy’s composite training data). Columns: RGB input, predicted normals, reconstructed depth, VLA grasp target (red cross) with object boundaries (cyan).

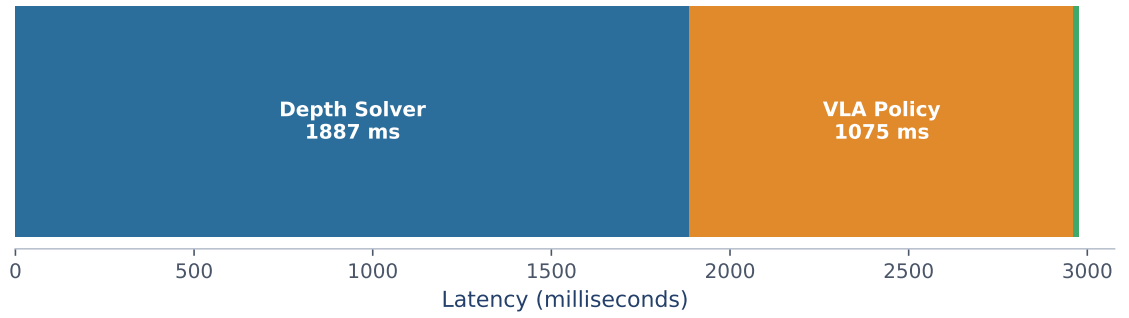


Figure 4.12: Latency breakdown across different stages of the pipeline.

4.10 Summary of Findings

The perception module recovers transparent-object geometry with a 19.6° mean normal error in-distribution. Out of distribution, it degrades slightly depending on object shape, yet the segmentation remains robust (RQ2). Given perfect normals, normal-guided depth reconstruction is *statistically indistinguishable* from a strong inpainting baseline ($p = 0.64$). For the evaluated shallow shapes, it performs identically to standard infilling. Therefore, the higher computational cost of normal guidance is not justified over the simpler method. The 0.21m gap between perfect and predicted normals evaluates how perception errors degrade metric depth estimates. This shows that improving normal accuracy is the highest-leverage upgrade in the geometry path (RQ3).

Three fine-tuning runs characterise the policy (RQ1, RQ5). Spatial accuracy remains low and data-limited across all three configurations. The overall on-target rates of 7.2%, 6.8%, and 2.7% sit at a statistically indistinguishable level on native frames, and no run bypasses the performance ceiling. Two structural findings recur in every run. First, native frames outperform composites (a significant +8.1 percentage-point difference in Run 2, with a clustered 95% CI of [1.1, 15.9], a trend that continues in Runs 3 and 4). This proves that realism of the compositing is the main constraint (Section 4.4.4). Second, a monotonic size gradient (0% on small objects, up to 14.1% on large objects) highlights encoder resolution as a secondary constraint.

The instruction robustness experiment shows that output formatting and semantic grounding are completely independent. Instruction diversification and a stronger training recipe fully repaired output validity (Run 4 achieved zero parse failures in 2,200 generations), but the targeting remained instruction-blind. Furthermore, a base-model control confirms that the fine-tuning process caused this blindness (the base model produced 4.0 different actions per frame, compared to 1.80 after fine-tuning).

Finally, a checkpoint selected by validation loss suffered from total generation failure. This confirms that standard validation loss is a misleading selection criterion for text-serialised action models (Section 4.7.1). End-to-end throughput runs at 0.34FPS, the CPU depth solver takes about 2.5s ($\approx 63\%$ of that time) rather than the 7B language model (RQ4).

Chapter 5

Discussion

This chapter interprets the findings of Chapter 4 against the research questions of Section 1.2 and the literature of Chapter 2. It examines the limitations of the study and draws out methodological lessons relevant beyond this specific system.

5.1 Synthetic Supervision and Spatial Grounding (RQ1)

The central question of this dissertation is whether composited synthetic data can teach a VLA to spatially ground transparent objects. The results support a qualified yes, limited by the data and not the model. The exact same architecture, optimiser, and budget produced two opposite outcomes. The defective configuration resulted in complete shortcut memorisation. In contrast, enforcing per-sample target diversity, background-identity splitting, and compositing-seam removal produced genuine image-conditioned predictions that accurately tracked target positions.

These results expose a hazard that the synthetic-data literature under-documents for *action* supervision (Section 2.6). When supervising a class label, shortcut learning degrades accuracy visibly. When predicting spatial coordinates, high aggregate token accuracy can hide shortcut learning. Standard evaluation metrics may fail to detect this issue unless per-coordinate errors are examined directly. The memorising policy scored 97.7% token accuracy because five of the seven action dimensions were constants, and its training loss reached exactly zero. A superficial review would pass both signals. The diagnostic that exposed this failure relied on measuring per-coordinate error on held-out data and treating a zero training loss as a warning sign. This diagnostic provides a transferable protocol for fine-tuning VLAs on procedurally generated data, complementing benchmark-level observations that VLA spatial competence must be engineered, not assumed [3, 24].

All three corrected runs converged to statistically indistinguishable validation losses and the same spatial-error floor near 20 bins, spanning a $16\times$ range in learning rate, two different LoRA target sets, and stopping schedules differing by an order of magnitude in step counts. The main limit is the data distribution, not the optimisation. Training on one object category, one instruction template, and 2D compositing without refraction or contact shadows strictly limits the spatial precision the policy can extract. This aligns with recent spatial-grounding literature, which improves localisation through richer supervision

or representations (such as dedicated pointing data [3] or explicit 3D encodings [17, 22]) rather than through longer training on identical data.

The two subsequent runs test this data-limitation claim directly. If an under-powered adaptation recipe caused the performance ceiling, strengthening the recipe should raise it. Run 4 applied the adaptation to all seven linear projections instead of restricting the updates to the query and value pair, raised the learning rate sixteen-fold, and doubled the LoRA scaling. These changes follow the official OpenVLA recipe [25] and target-module studies showing that MLP projection adaptation outperforms attention-only adaptation [19, 25].

This recipe change had a significant effect on the *form* of the output. Off-template generation evolved from unstructured, multilingual tokens (Run 2), to degraded-but-near-valid coordinate strings (Run 3), to fully valid tokens under all eleven instruction conditions (Run 4, achieving zero parse failures in 2,200 generations). Despite this format recovery, the spatial ceiling remained static. Run 3 grasped at 6.8% overall. Run 4 grasped at 2.7% overall, scoring 4.5% [2.2, 9.0] on native frames, which is statistically indistinguishable from Run 2’s 7.2%.

The native subset spatially grounds the model because it contains the necessary transparency cues. Run 4’s overall figure falls slightly below the others only because its composite subset drops to 0.7%, a within-noise fluctuation near the performance floor, not a recipe-induced regression. Because formatting metrics and spatial accuracy responded independently to a change in recipe, the performance ceiling must be viewed as an inherent property of the training data distribution, meaning it is not a limitation of model capacity.

Furthermore, the corrected baseline comparison shows the policy beats a centre-prior on the *median* distance but fails on the *mean*, with the gap consisting of a $\sim 26\%$ tail of gross mislocalisations. Spatial grounding is therefore real but non-uniform: it works on the majority of samples but fails completely on a substantial minority. Instead of finding a reliable coordinate, the model just outputs a generic spatial pattern. This metric-space behaviour mirrors the format-versus-grounding separation established during the instruction-shift evaluation.

5.2 Robustness Is Not Grounding

The instruction-robustness evaluation (Section 4.5) demonstrates that a model emitting valid, well-formed actions has not necessarily “learned the task.” The recipe-corrected policy produces zero malformed outputs across paraphrased, absent-object, and null instructions, yet places its target in the same spatial distribution regardless of the prompt. Output validity reached 100% while instruction sensitivity remained absent. Formatting the output and understanding the scene are two different things. The model learned the format, but failed at the task.

The base-model control (Section 4.6) pinpoints the cause of this failure. On real frames, the un-fine-tuned OpenVLA-7B model emits four distinct actions across different instructions, whereas the fine-tuned policy emits fewer than two. The instruction was informative to the base model but became inert during fine-tuning. Because every training target in

this specific task depended solely on the image, language was ignored by the model during fine-tuning. Fine-tuning on a narrow, image-based objective degrades the base model’s existing instruction-following capabilities.

The catastrophic-forgetting literature describes this exact failure mode at the benchmark level [27?]. Fine-tuning on a narrow, image-determined objective degrades an instruction-following capability the base model possessed, even as in-task metrics improve. The remedy requires supervision where the instruction is causally necessary to the answer. Training on multiple objects while varying the target forces semantic grounding instead of relying on it to develop on its own.

Regarding checkpoint selection, the adapter with the lowest validation loss computed against the ground truth in Run 4 failed to generate a parseable action in almost every case (Section 4.7.1). standard validation loss rewards next-token agreement given the ground-truth prefix without measuring whether free-running generation adheres to the action grammar. For action models serialising output as text, these two metrics diverge to the point where the loss-optimal checkpoint fails completely in deployment. Consequently, behavioural evaluation is a strict requirement for checkpoint selection, rendering validation-loss curves without a corresponding generation-quality metric highly misleading for this model class.

5.3 Perception Generalisation and Error Propagation (RQ2, RQ3)

The perception results reproduce the pattern enabling ClearGrasp [2]. RGB-only networks recover transparent-object geometry well within their training distribution, and their segmentation transfers to novel shapes far better than their normal estimation. The shape-dependence of the degradation (a 6° loss on a geometrically similar unseen object versus 14.5° on a dissimilar one) indicates that broadening the shape distribution of training data matters more for geometric generalisation than increasing data volume on a single shape.

The ablation identifies perception-to-depth error propagation quantitatively. Because the solver driven by perfect normals is statistically indistinguishable from a strong inpainting baseline ($p = 0.64$), the 0.21 m RMSE deficit of the deployed configuration stems from the 19.55° normal error. This result proves that normal accuracy is the highest-leverage improvement in the geometry path, not solver design.

The finding that standard inpainting is competitive on these objects clarifies operating boundaries. ClearGrasp’s evaluation objects are shallow relative to their surroundings, meaning boundary interpolation recovers most of the depth field. The normal-guided machinery earns its computational complexity only where surfaces curve substantially. Reporting this boundary condition delineates where a ClearGrasp-style pipeline is necessary versus where a simple infilling baseline suffices.

5.4 From Spatial Accuracy to Grasp Feasibility (RQ4)

The grasp results show that the first-generation system failed at a 2.6% on-target rate due to three specific data defects: 10% invalid training labels, missing transparency cues in composited images, and destructive double downsampling. The second iteration corrected all three defects, nearly tripling the on-target rate to 7.2%. Subsequent runs (Section 5.1) tested whether instruction diversity or a stronger adaptation recipe could raise this ceiling further, but neither succeeded.

Evaluating the performance split by sample type reveals that on native ClearGrasp frames (containing true refraction, cast shadows, and specular highlights), the model achieves an 11.3% on-target rate and 20.2% on-object rate. On composites lacking these cues, it drops to 3.1% and 5.5% respectively. This four-to-one ratio identifies compositing fidelity as the dominant constraint on spatial precision, overriding model architecture, LoRA configurations, and training budgets. Under the pre-registered significance bootstrapping across scenes test, the difference is +8.1 percentage points (95% CI [1.1, 15.9]), resting on a pre-registered significance test. Furthermore, the composite performance in Run 2 closely matches the full-dataset performance of Run 1 (2.6%), indicating that label and resolution fixes contributed marginally compared to authentic visual cues.

The size breakdown provides a secondary constraint: 0% on-target for objects below 142 pixels, compared to 14.1% for objects above 562 pixels. At the 224×224 working resolution, small transparent objects occupy too few encoder patches to form a reliable localisation signal. Higher processing resolution would likely yield further gains, at the cost of deviating from OpenVLA’s pre-trained encoder configuration.

These findings carry a practical implication for synthetic-only VLA training targeting transparent objects: simple compositing is insufficient. The visual cues distinguishing transparent localisation from opaque localisation (background refraction distortion, caustic patterns, and specular geometry) are precisely what compositing deletes. Physically based rendering assets or real captures must be mandatory training components.

The 7.2% overall on-target rate remains far below deployment requirements, but the three training runs help clarify where the problem is. Run 4’s stronger, literature-aligned recipe showed no improvement in the success rate. By elimination and by the native-versus-composite evidence, the real limit lies in the training data: how realistic it is, how many object categories it covers, and effective resolution. Whether this gap requires higher-fidelity data or a dedicated spatial-prediction architecture such as RoboPoint [3] remains open for future work, but the recipe axis has been tested and proved insufficient.

Finally, the latency of the 7B language model accounts for roughly 36% of the 2.98s frame time, while the CPU-bound sparse depth solver accounts for roughly 63%. Given the ablation finding that inpainting matches the solver on these shallow objects, replacing the solver with a single `cv2.inpaint` call halves the per-frame cost with no measured loss in depth accuracy.

5.5 Limitations

The following limitations bound the scope of this work. First, all policy evaluations rely on synthetic composites. No real-camera or physical-robot validation was performed, leaving the sim-to-real gap for the *policy* unquantified. The perception module inherits Clear-Grasp’s evidence of real-world transfer, but this transfer was not re-verified independently.

Second, generating valid action tokens and understanding semantic instructions are independent properties. While a stronger training recipe fixed the syntax of the predictions, the model still ignored the meaning of the text prompts.

Third, the in-distribution perception figures overlap with the training data and serve strictly as performance upper bounds, with generalisation claims relying on the out-of-distribution categories.

Fourth, the grasp proxy uses a geometric criterion rather than a physics simulation, completely ignoring gripper geometry, object slip, and approach kinematics.

Fifth, checkpoint-level policy evaluations used 50 samples per checkpoint for computational tractability, while headline grasp measurements evaluated 251 samples across 78 scenes.

Sixth, the action space locks height and orientation to presume top-down grasps on a planar belt, omitting full 6-DOF grasp synthesis.

Seventh, validation source-frames are near-duplicates of training frames (1,743 out of 1,755 source frames sit within two frames of a training instance). Consequently, in-distribution accuracies represent optimistic ceilings. Comparative findings (such as native versus composite performance, the size gradient, and run-to-run deltas) rely on within-model contrasts and survive this leakage, but absolute performance rates should not be interpreted as true held-out generalisation.

Eighth, actions are serialised as text tokens instead of being processed through Open-VLA’s native continuous action head. The off-template format fragility documented in Chapter 4 stems partly from this serialisation and may not apply to continuous-action variants.

5.6 Summary

This study centres on three primary findings. First, the training distribution dictates whether spatial learning occurs at all. The exact same architecture and budget produced either shortcut memorisation or genuine image-conditioned prediction depending solely on the data pipeline, and no training recipe intervention could overcome the spatial ceiling set by how realistic the data is.

Second, output formatting robustness and semantic grounding are two independent properties. A stronger training recipe completely repaired the format of the policy’s predictions while not improving instruction understanding, a failure caused by fine-tuning on a task that never strictly required the model to read the instruction to locate the target.

Third, perception error represents the dominant and quantified cost in the geometry path, translating directly into a measurable error in metres. The distance to deployable

grasp precision is now quantifiable and attributable to identifiable data-fidelity gaps instead of component failures. Overall, this work delivers a diagnosed explanation of why this synthetic-VLA recipe fails on transparent objects, supported by a controlled experimental arc demonstrating which levers move performance and which do not.

Chapter 6

Conclusions and Recommendations

6.1 Conclusions

This dissertation evaluated a cascaded framework combining procedural synthetic data generation, geometric depth reconstruction, and Vision-Language-Action fine-tuning for transparent plastic sorting. The findings demonstrate that while the perception and geometry stages perform reliably, policy-level spatial grounding is limited by how realistic the training data is, not by the optimisation recipe. With respect to the research questions, the work concludes:

RQ1 (synthetic supervision). Composited synthetic data alone is insufficient for precise transparent-object localisation. The first-generation policy achieved only a 2.6% on-target grasp placement. Correcting three data defects (invalid labels, missing transparency cues, and redundant downsampling) and mixing authentic ClearGrasp renders at roughly 50% nearly tripled the on-target rate to 7.2%. A diagnostic type breakdown (11.3% on native frames versus 3.1% on composites; a significant difference of +8.1 percentage points, clustered 95% CI [1.1, 15.9]) attributes this improvement to restoring refraction, shadow, and specular cues. Two subsequent fine-tuning runs tested this data limit: neither instruction diversification (Run 3, 6.8% on-target) nor a stronger, literature-aligned LoRA recipe (Run 4, 2.7% overall) raised the spatial ceiling, with all three runs performing at a statistically indistinguishable level on native frames. Data pipeline design determines whether VLA spatial learning occurs. Data realism (the presence of transparency-specific visual physics) sets the performance ceiling, and no adaptation recipe can move it. A matched-sample comparison against a centre-prior baseline further shows that spatial grounding is non-uniform: the policy beats the baseline on median distance (42.2 vs. 47.2 px) but not on mean, owing to a 26% tail of gross mislocalisations.

RQ2 (transparent-object perception). An RGB-only network recovers transparent-object geometry with a 19.6° mean surface-normal error and 91% segmentation IoU within its training category. On unseen categories, segmentation degrades modestly ($\sim 77\%$ IoU), while normal estimation degrades in proportion to geometric dissimilarity from the training shapes (25.5° on a similar unseen object versus 34.1° on a dissimilar one). What determines geometric generalisation is broader shape diversity in the training data, not just more volume.

RQ3 (depth reconstruction). With perfect normal input, the Poisson solver is statistically indistinguishable from a strong inpainting baseline (0.129 m vs. 0.123 m RMSE; median difference -0.1 mm, $p = 0.64$), validating its implementation. With predicted normals, error increases to 0.338 m. The 0.21 m gap between perfect and predicted normals quantifies how perception error propagates into metric depth, which identifies normal accuracy, not solver design, as the highest-leverage improvement in the geometry path. For shallow objects, standard inpainting is statistically equivalent and far cheaper. Using normal-guided reconstruction is only worth the computational cost if the objects are deeply curved.

RQ4 (computational feasibility). The integrated pipeline runs at 0.34 frames per second on an A100. It is bounded not by the 7-billion-parameter language model ($\sim 36\%$ of frame time) but by the CPU sparse depth solver ($\sim 63\%$). Replacing the solver with an equivalent inpainting path would roughly halve per-frame costs. This rate sits well below conveyor speeds; real-time operation was out of scope for this feasibility study, and profiling attributes computational cost. It does not claim the system is deployable. Furthermore, task feasibility is constrained: on-target grasp placement peaks at 7.2% and is heavily dependent on object size.

RQ5 (instruction grounding). The policy does not condition its spatial predictions on the content of the language instruction. Across a three-run arc, instruction diversification and a stronger recipe progressively repaired the format of the output: off-template multilingual gibberish in Run 2, near-valid degraded coordinate strings in Run 3, and zero malformed outputs across eleven instruction conditions in Run 4 across 0/2,200 generations. However, the on-target rate stayed flat at 4–8% across every condition, including absent-object and null instructions. Output validity became total while instruction sensitivity remained absent. A base-model control confirms this collapse comes directly from fine-tuning: unadapted OpenVLA emits 4.0 distinct actions across distinct instructions per frame, whereas the fine-tuned policy emits 1.80. Furthermore, output syntax and semantic grounding are separable properties, and only the former was repairable without better data. Finally, standard validation loss proved to be an unsafe checkpoint-selection signal for text-serialised action models, as the loss-optimal checkpoint suffered total generation failure.

6.2 Contributions to Knowledge and Practice

This work diagnoses shortcut memorization in composited action supervision and establishes the metrics required to expose it. Furthermore, analyzing the synthetic-only route defines the essential data controls for future robotic sorting, specifying exactly where simple inpainting baselines succeed or fail.

6.3 Recommendations and Future Work

The findings motivate the following directions, ordered by expected return, with each tied to a measured failure:

1. **Data fidelity (the proven primary lever).** Replace alpha compositing with physically based rendering, or augment composites with synthetic refraction and contact shadows, so that object position is once again linked to transparency-specific appearance. Both the policy’s precision ceiling (shown by the significant native-versus-composite gap) and the perception module’s out-of-distribution degradation trace to this gap, which no recipe or optimisation change addressed.
2. **Instruction-necessary supervision.** The grounding failure in RQ5 is a property of the training signal, not the architecture. Because every target in this setup was determined by the image alone, the language channel provided no useful gradient and was consequently ignored. Supervision in which the instruction is causally required to disambiguate the target—such as multiple co-present objects with instruction-selected grasp points across the full ClearGrasp category set—provides the direct remedy, forcing semantic grounding instead of hoping for it.
3. **Solver acceleration.** Port the sparse depth reconstruction to GPU, or amortise it into the perception network as a depth-completion head, to remove its $\sim 63\%$ latency share. Alternatively, deploy the inpainting path, which matches the solver on these shallow objects at zero additional optimization cost.
4. **Continuous-action serialisation.** The off-template format fragility is partly an artefact of serialising actions as text tokens. Adopting a continuous action head (as in the OFT recipe [25]) would decouple motor-output validity from language-model generation stability, and its FiLM conditioning offers a further defence for the language channel identified in recommendation 2.
5. **Depth as policy input.** The policy currently never receives the reconstructed depth. Feeding the depth map directly would allow the model to reason about the 3D structure recovered by the perception path, instead of predicting a 2D point that a separate stage must lift into 3D.
6. **Physical and simulation validation.** Because physical robot access was unavailable during this project, future work must close the loop in staged steps: capturing real RGB-D data of transparent objects to measure the policy’s sim-to-real gap, verifying open-loop reprojection accuracy in millimetres against a calibrated camera-to-robot transform, simulating the delta cell as an intermediate environment, and only then conducting live picking trials.

6.4 Closing Remark

This project highlights the importance of rigorous data validation in VLA fine-tuning. By diagnosing the model’s failures, this work provides a clear framework for future attempts to train VLAs on synthetic data.

The data pipeline determines whether spatial learning happens, and how realistic the data is sets the precision ceiling, not the adaptation method. Output format and semantic

grounding are independent properties, while format errors were repairable; semantic errors were not. Perception errors reduce depth accuracy by 0.21 m and throughput is restricted by the solver rather than the language model.

Several early observations in this project were corrected once their measurement protocols were found to be unsound. Consequently, the systematic diagnosis of these hidden errors serves as a key outcome of this research.

Bibliography

- [1] Dina Bashkirova, Mohamed Abdelfattah, Ziliang Zhu, James Akl, Fadi Alladkani, Ping Hu, Vitaly Ablavsky, Berk Calli, Sarah Adel Bargal, and Kate Saenko. ZeroWaste dataset: Towards deformable object segmentation in cluttered scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21147–21157, 2022.
- [2] Shreeyak Sajjan, Matthew Moore, Mike Pan, Ganesh Nagaraja, Johnny Lee, Andy Zeng, and Shuran Song. Clear grasp: 3d shape estimation of transparent objects for manipulation. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 3634–3642. IEEE, 2020.
- [3] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.
- [4] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [5] Brianna Zitkovich, Tianhe Yu, Sichun Goldsack Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the Conference on Robot Learning (CoRL)*, pages 2165–2183. PMLR, 2023.
- [6] Maria Koskinopoulou, Fredy Raptopoulos, George Papadopoulos, Nikitas Mavrakis, and Michail Maniadakis. Robotic waste sorting technology: Toward a vision-based categorization system for the industrial robotic separation of recyclable waste. *IEEE Robotics & Automation Magazine*, 28(2):50–60, 2021.
- [7] Cesar Lubongo, Mohammed AA Bin Daej, and Paschalis Alexandridis. Recent developments in technology for sorting plastic for recycling: The emergence of artificial intelligence and the rise of the robots. *Recycling*, 9(4):59, 2024.
- [8] Edgar Ramos, Arminda Guerra Lopes, and Fábio Mendonça. Application of machine learning in plastic waste detection and classification: A systematic review. *Processes*, 12(8):1632, 2024.

- [9] Nils Kroell, Xiaozheng Chen, Kathrin Greiff, and Alexander Feil. Optical sensors and machine learning algorithms in sensor-based material flow characterization for mechanical recycling processes: A systematic literature review. *Waste management*, 149:259–290, 2022.
- [10] Ranjan Sapkota, Yang Cao, Konstantinos I Roulmeliotis, and Manoj Karkee. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv preprint arXiv:2505.04769*, 2025.
- [11] Kento Kawaharazuka, Jihoon Oh, Jun Yamada, Ingmar Posner, and Yuke Zhu. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 2025.
- [12] Janusz Bobulski and Mariusz Kubanek. Deep learning for plastic waste classification system. *Applied Computational Intelligence and Soft Computing*, 2021(1):6626948, 2021.
- [13] Ting-Wei Wu, Hua Zhang, Wei Peng, Fan Lü, and Pin-Jing He. Applications of convolutional neural networks for intelligent waste identification and recycling: A review. *Resources, Conservation and Recycling*, 190:106813, 2023.
- [14] Gary White, Christian Cabrera, Andrei Palade, Fan Li, and Siobhan Clarke. Wastenet: Waste classification at the edge for smart bins. *arXiv preprint arXiv:2006.05873*, 2020.
- [15] Jian Yang, Yu-Peng Xu, Pu Chen, Jing-Yan Li, Dan Liu, and Xiao-Li Chu. Combining spectroscopy and machine learning for rapid identification of plastic waste: Recent developments and future prospects. *Journal of Cleaner Production*, 431:139771, 2023.
- [16] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- [17] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- [18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [19] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.
- [20] Y. Zhong et al. A survey on vision-language-action models: An action tokenization perspective. *arXiv preprint arXiv:2507.01925*, 2025.

- [21] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
- [22] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-vla: A 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.
- [23] Heyu Guo, Shanmu Wang, Ruichun Ma, Shiqi Jiang, Yasaman Ghasempour, Omid Abari, Baining Guo, and Lili Qiu. Omnivla: Physically-grounded multimodal vla with unified multi-sensor perception for robotic manipulation. *arXiv preprint arXiv:2511.01210*, 2025.
- [24] Zheng Cai, Pierre-Louis Guhur, Cordelia Schmid, and Ivan Laptev. SpatialBench: Benchmarking spatial cognitive capabilities of vision-language-action models. *arXiv preprint arXiv:2406.13537*, 2024.
- [25] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [26] Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *IEEE Transactions on Audio, Speech and Language Processing*, 2025.
- [27] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, et al. Lora learns less and forgets less. *arXiv preprint arXiv:2405.09673*, 2024.
- [28] Mohammad Shahin, F Frank Chen, Mazdak Maghanaki, Hamed Mehrzadi, and Ali Hosseinzadeh. Toward sustainable production: a synthetic dataset framework to accelerate quality control via generative and predictive ai. *The International Journal of Advanced Manufacturing Technology*, 138(11):5979–6018, 2025.
- [29] Debidatta Dwibedi, Ishan Misra, and Martial Hebert. Cut, paste and learn: Surprisingly easy synthesis for instance detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 1310–1319. IEEE, 2017.
- [30] Nikita Dvornik, Julien Mairal, and Cordelia Schmid. Modeling visual context is key to augmenting object detection datasets. In *European Conference on Computer Vision*, pages 375–391. Springer, 2018.
- [31] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017.